Progressive Education Society's
Modern College of Engineering, Pune - 411005
Department of Electronics & Computer

AY-2024-25 Semester - I

Microcontroller and Applications

Course In-charge
Mrs. R. S. Chaudhari

Unit IV

MSP430 Microcontroller Architecture and Low Power Features

Syllabus

- Low Power 16-bit MSP430x5xx microcontroller architecture,
- address space,
- on-chip peripherals (analog and digital),
- register sets,
- instruction set,
- instruction formats,
- various addressing modes of MSP430 devices;
- variants of the MSP430 family viz. MSP430x2x, MSP430x4x, MSP430x5x and their targeted applications,
- system clocks,
- low Power aspects of MSP430: low power modes, Active vs Standby current consumption, FRAM vs Flash for low power; reliability.

Introduction to MSP430

- MSP430 fits between traditional 8- and 16-bit processors. The 16-bit data bus and registers clearly define it as a 16-bit processor
- It can address only 2^{16} = 64KB of memory
- Another feature of the MSP430 that stems from its recent introduction is that it is designed with compilers in mind
- Most small microcontrollers are now programmed in C, and it is important that a compiler can produce compact, efficient code
- MSP430 has 16 registers in its CPU, which enhances efficiency because they can be used for local variables, parameters passed to subroutines, and either addresses or data. This is a typical feature of a RISC, but unlike a “pure” RISC, it can perform arithmetic directly on values in main memory
- Microcontrollers typically spend much of their time on such operations

Introduction to MSP430

- MSP430 is a microcontroller family from Texas Instruments
- It is one of the simplest microcontroller families from TI. Its more powerful siblings include the TMS470, which is based on the 32/16-bit ARM7
- ‘MSP’ stands for Mixed Signal Processor (ICs that contain both analog and digital circuitry)
- CPU is small and efficient, with a large number of registers
- It is extremely easy to put the device into a low-power mode. No special instruction is needed. The mode is controlled by bits in the status register
- The MSP430 is awakened by an interrupt and returns automatically to its low-power mode after handling the interrupt

Introduction to MSP430

- There are several low-power modes, depending on how much of the device should remain active and how quickly it should return to full-speed operation
- There is a wide choice of clocks. Typically, a low-frequency watch crystal runs continuously at 32 KHz and is used to wake the device periodically.
- The CPU is clocked by an internal, digitally controlled oscillator (DCO), which restarts in less than 1µs in the latest devices
- Therefore the MSP430 can wake from a standby mode rapidly, perform its tasks, and return to a low-power mode.
- A wide range of peripherals is available, many of which can run autonomously without the CPU for most of the time.

Introduction to MSP430

- Many portable devices include liquid crystal displays, which the MSP430 can drive directly.
- Some MSP430 devices are classed as application-specific standard products (ASSPs) and contain specialized analog hardware for various types of measurement.
- The series has more than 500 different microcontrollers.
- MSP430 series support low power operation, over 5 low power modes are available (Active, Sleep-5, Power down)
- It can be used for
  - General purpose sensing and measurement - a single enhanced sensor can indirectly monitor a large context, without direct instrumentation of objects
  - Capacitive touch sensing - is a technology, based on capacitive coupling, that can detect and measure anything that is conductive. They are found in many of our electronic devices, seamlessly integrated into our smartphones, tablets and other products that feature touch screens.
  - Ultrasonic sensing - measures the distance to an object using ultrasonic sound waves

Difference between 8051 and MSP430

8051	MSP430
8-bit micro-controller	16-bit micro-controller
128 bytes of internal RAM. But modern variants of 8051 have 256 bytes of RAM	28/256/512 Bytes RAM/SRAM
4KB of internal ROM and can be extended upto 64KB	2/4/8/16 KB of ROM also offers upto 32 KB
No word instruction in general byte instruction	Word and byte instructions
Some special bit instructions for a limited address range	No bit instruction. Equivalent bit instruction can be obtained by use of a mask
Handles only digital signal	Handles mixed signals – analog and digital
Speed is 12 clock/instruction cycle	Speed is 6 clock/instruction cycle
Power consumption is average	Power consumption is ultra low
No power saving modes	5 power saving modes
Multiplications and MAC operations are done in software	Hardware multiplier and MAC units are present

Difference between 8051 and MSP430

8051	MSP430
Families include 8051 variants	Families include MSP430X, MSP430FR57xx, MSP430x1xx to x6xx series.
Cost is very low	Cost is average
Popular Micro-controllers AT89C51, P89v51, etc	Popular Micro-controller MSP430G2553, MSP430 launchpad
Manufacturers are NXP, Atmel, Silicon Labs, Dallas, Cyprus, Infineon, etc	Manufacturer is Texas Instrument
CPU Speed is typically up to 33 MHz	CPU Speed is typically up to 25 MHz
Instruction Set is Limited	Instruction Set is More complex and diverse
On-chip memory is Limited	On-chip memory is More and larger
On-chip peripherals is Limited	More and diverse on-chip peripherals
Interrupt handling is Simple and limited	Interrupt handling is More complex and efficient
Development tools are Widely available and mature	Development tools are Growing ecosystem

MSP430 FAMILY

MSP430 SERIES	MSP430 FAMILY	FREQUENCY	MEMORY	GPIO
CAPACITIVE SENSING MCUs	FR25x/FR26x	16MHz	FRAM: UP TO 16KB SRAM: UP TO 4KB	15-19
VALUE LINE SENSING MCUs	FR2XX/FR4X	16MHz	FRAM: UP TO 16KB SRAM: UP TO 4 KB	16-64
	G2X/I2X	16MHz	FLASH: UP TO 56KB SRAM: UP TO 4KB	4-32
PERFORMANCE -SENSING MCUs	FR5X/FR6X	16MHz	FRAM: UP TO 256KB SRAM: UP TO 8KB	17-83
	F5X/F6X	25MHz	FLASH: 512KB SRAM: UP TO 67 KB	29-90
OTHER MSP430 MCUs	F2X/F4X	16MHz	FLASH: UP TO 120KB SRAM: UP TO 8KB	14-80
	F1X	16MHz	FLASH: UP TO 120KB SRAM: UP TO 10KB	10-48

Features of MSP430G2x53

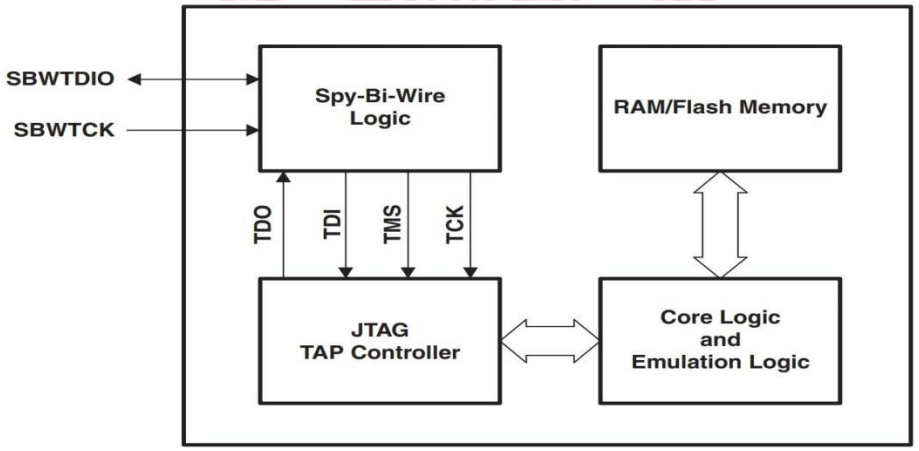
- Low Supply-Voltage Range: 1.8 V to 3.6 V
- Ultra-Low Power Consumption
  - Active Mode: 230 μA at 1 MHz, 2.2 V
  - Standby Mode (LPM): 0.5 μA
  - Off Mode (RAM Retention): 0.1 μA
- Five Power Saving Modes (LPM0 -LPM4)
- Ultra-Fast Wake-Up From Standby Mode in Less Than 1 μs
- 62.5ns Instruction Cycle Time (register to register operation)
- Basic Clock Module Configurations
  - Internal Frequencies up to 16 MHz with four calibrated frequencies - 1MHz, 8Mhz, 12Mhz, 16Mhz.
  - Internal Very-Low-Power Low Frequency (LF) Oscillator.
  - 32-kHz Crystal as external clock source
  - External Digital Clock Source
- Two 16-Bit Timer_A With Three Capture/Compare Registers
  - A timer is a hardware counter. Since 16 bit it counts from 0 to 65535.

Features of MSP430G2x53

- Up to 24 Capacitive-Touch Enabled I/O Pins
  - Inbuilt capacitive touch feature on all GPIO Pins
- Universal Serial Communication Interface (USCI)
  - UART
  - IrDA Encoder and Decoder - is a CMOS device which encodes and decodes bit data in conformance with the IrDA specification
  - Synchronous SPI (Serial Peripheral Interface)
  - I2C (Inter IC Communication)
- On-Chip Comparator
- Analog-to-Digital Cycle Time (A/D) Conversion
- 10-Bit 200-ksps Analog-to-Digital (A/D) Converter
  - Internal Reference(1.5 V or 2.5 V)
  - Sample and Hold with programmable sample periods
  - Eight External Input channels
- Brownout Detector
- Serial Onboard Programming
- No External Programming Voltage Needed
- Programmable Code Protection by Security Fuse

Features of MSP430G2x53

- On-Chip Emulation Logic With Spy-Bi-Wire - Spy-Bi-Wire (SBW) is a serial JTAG protocol that allows a host to access the memory of a target MSP430 microcontroller using only two connections instead of the usual four.
- Spy-Bi-Wire is a serialized JTAG protocol. The two connections are a bidirectional data output (SBWTDIO), and a clock (SBWTCK). The clocking signal is split into a period of three clock pulses, for each clock pulse the TDI, TDO and TMS signals are passed on the microcontroller via the bidirectional data output.

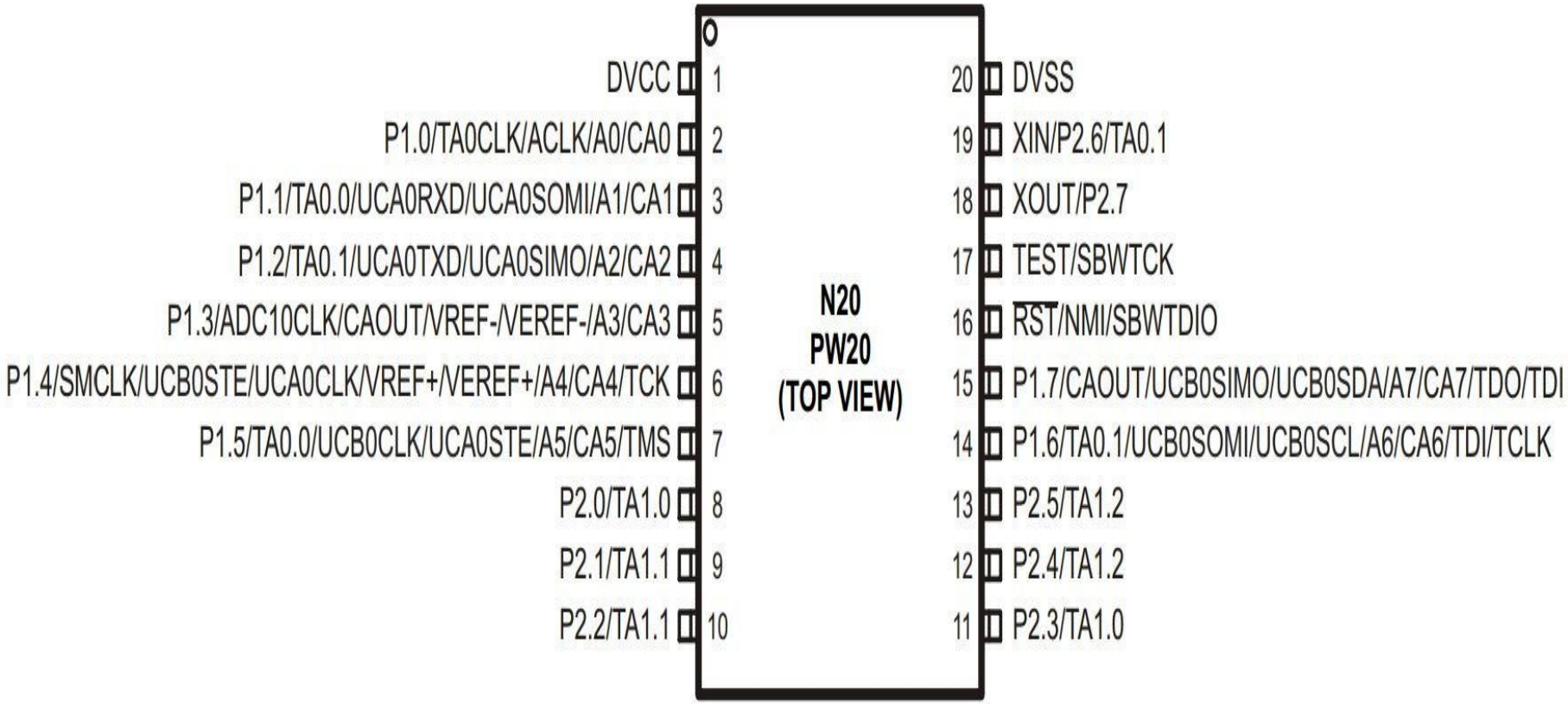


Spy-Bi-Wire Basic Concept Diagram

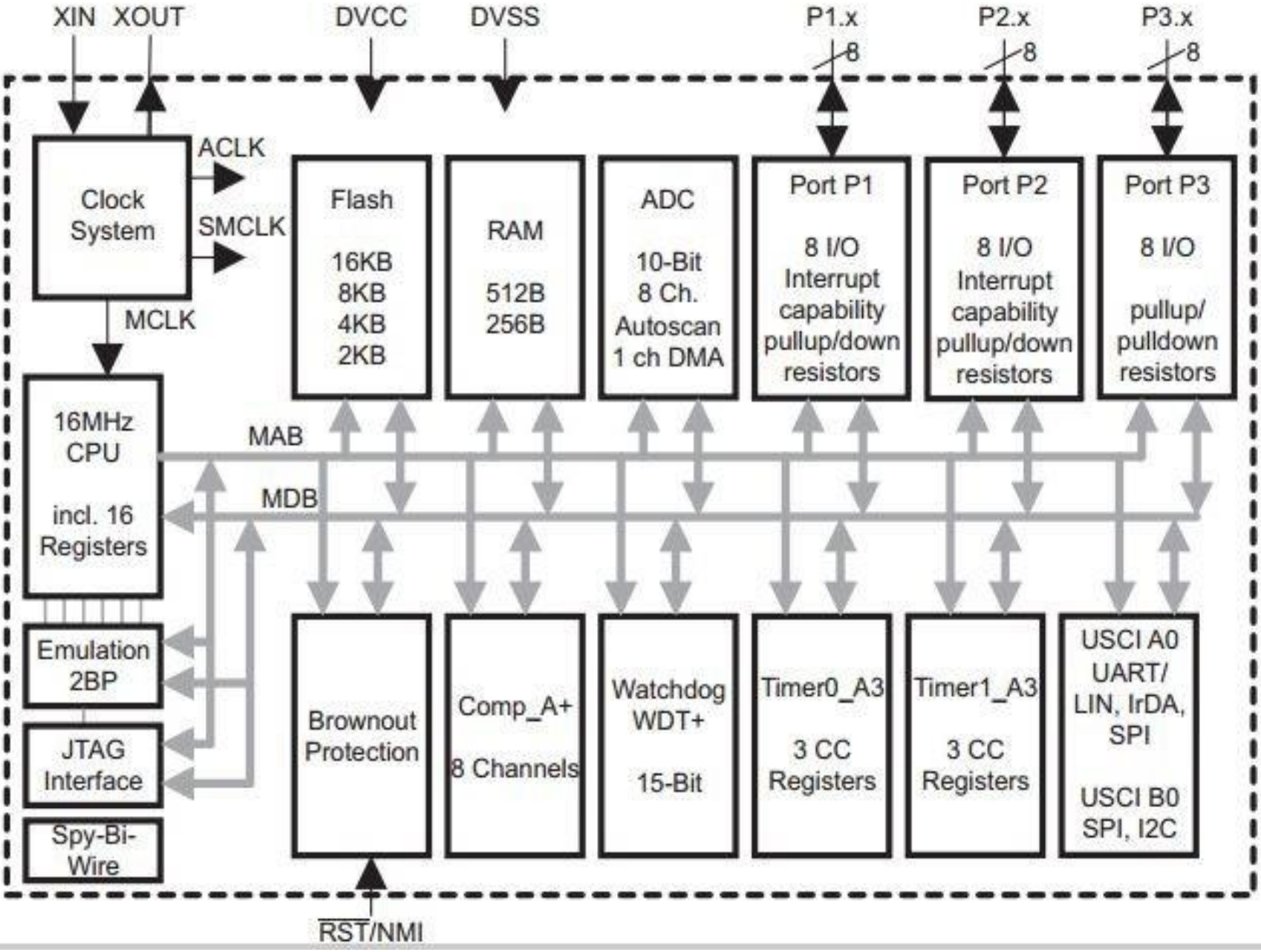
- The SBWTCK signal is the clock signal and is a dedicated pin. In normal operation, this pin is internally pulled to ground.
- The SBWTDIO signal represents the data and is a bidirectional connection. To reduce the overhead of the 2-wire interface, the SBWTDIO line is shared with the RST/NMI pin of the device.

MSP430 PINOUT

Device Pinout, MSP430G2x13 and MSP430G2x53, 20-Pin Devices, TSSOP and PDIP



Functional Block Diagram, MSP430G2x53



Introduction to the MSP430 CPU

- RISC Architecture with 27 core instructions and 7 addressing modes.
- 16-Bit Address Bus and 16-Bit Data Bus
- A set of 16 16-Bit Registers reduces fetches to memory
- Maximum clock frequency:- 16MHz(G-Series) 25MHz(F-Series)
- Constant generator provides six most used immediate values and reduces code size.
- Direct memory-to-memory transfers without intermediate register holding.
- Word and byte addressing and instruction formats.

Introduction to the MSP430 CPU

- MSP430 has Von Neumann Memory Architecture. There is only one set of addresses which cover both volatile and non-volatile memories.
- Address Bus is 16 bits wide, so 2^16 = 65,536 = 64K addresses. So, Address Range is 0x0000 to 0xFFFF.
- MSP430X has an Address Bus of 20 Bits, so 2^20 = 10,48,576 addresses are possible.
- The memory data bus is 16 bits wide and can transfer either a word of 16 bits or a byte of 8 bits.
- The address of a word is defined to be the address of the byte with the lower address, which must be even. Eg: the two bytes at 0x4000 and 0x4001 can be fetched as a word with address 0x4000 in a single cycle.
- Instructions are composed of words and therefore must lie at even addresses.

Memory Architecture

- Words are stored as two bytes in the memory in the little endian ordering
- In little endian ordering, the lower order byte is stored at the lower address and the higher order byte at higher address.

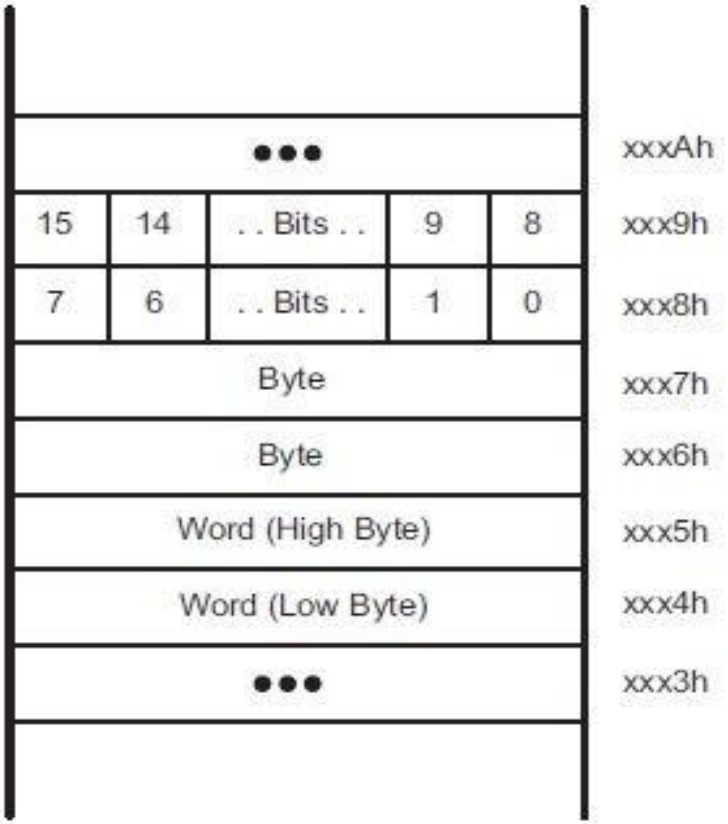


Figure 1-3. Bits, Bytes, and Words in a Byte-Organized Memory

MSP430 G Series Memory Map

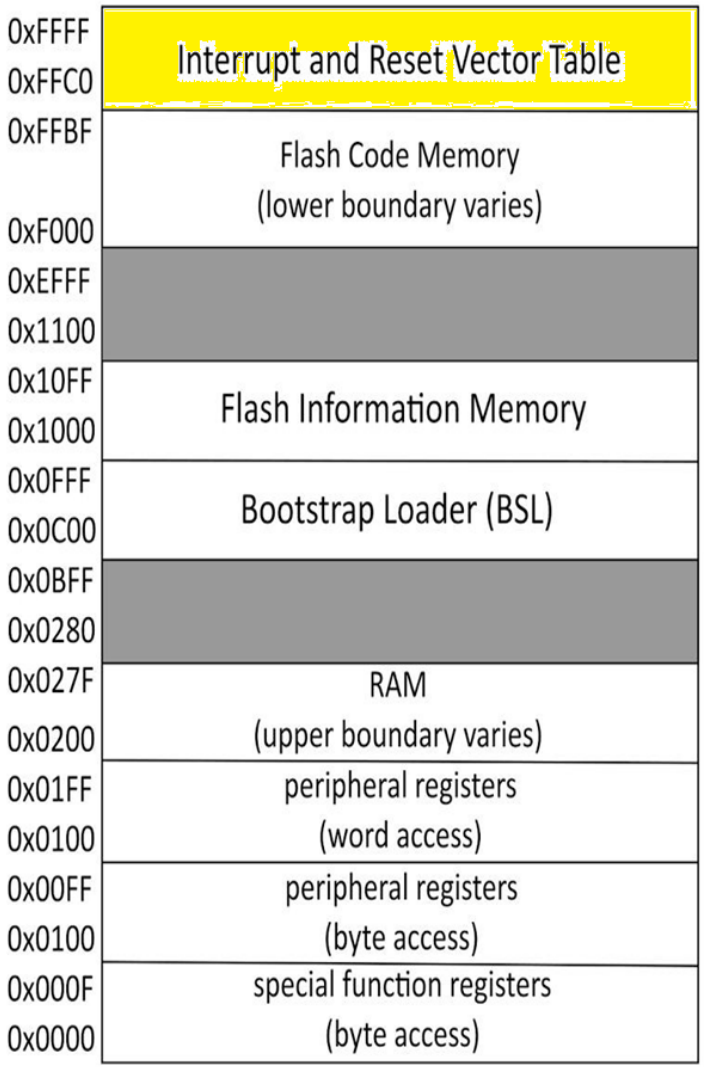
		MSP430G2153	MSP430G2253 MSP430G2213	MSP430G2353 MSP430G2313	MSP430G2453 MSP430G2413	MSP430G2553 MSP430G2513
Memory	Size	1kB	2kB	4kB	8kB	16kB
Main: interrupt vector	Flash	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0
Main: code memory	Flash	0xFFFF to 0xFC00	0xFFFF to 0xF800	0xFFFF to 0xF000	0xFFFF to 0xE000	0xFFFF to 0xC000
Information memory	Size	256 Byte	256 Byte	256 Byte	256 Byte	256 Byte
	Flash	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h
RAM	Size	256 Byte	256 Byte	256 Byte	512 Byte	512 Byte
		0x02FF to 0x0200	0x02FF to 0x0200	0x02FF to 0x0200	0x03FF to 0x0200	0x03FF to 0x0200
Peripherals	16-bit	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h
	8-bit	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h
	8-bit SFR	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h

Memory Organization (Flash)

- MSP430 flash memory is partitioned into segments. A segment is the smallest size of flash memory that can be erased.
- The flash memory is partitioned into main and information memory sections.
- The differences between the two sections are the segment size and the physical addresses. The information memory has four 64-byte segments. The main memory has one or more 512-byte segments.

Memory Organization (Flash)

- The start address of Flash/ROM depends on the amount of Flash/ROM present and varies by device.
- The end address for Flash/ROM is 0xFFFF
  for devices with less than 60KB Flash/ROM.
- Flash can be used for both code and data but is generally used as code memory. The code memory holds the program.
- The interrupt vectors are used to handle the interrupts. The interrupt vector table is mapped into the upper 16 words of Flash/ROM address space, with the highest priority interrupt vector at the highest Flash/ROM word address (0xFFFE).



8.4 Interrupt Vector Addresses

The interrupt vectors and the power-up starting address are located in the address range 0FFFFh to 0FFC0h. The vector contains the 16-bit address of the appropriate interrupt handler instruction sequence.

If the reset vector (located at address 0FFFEh) contains 0FFFFh (for example, flash is not programmed), the CPU goes into LPM4 immediately after power-up.

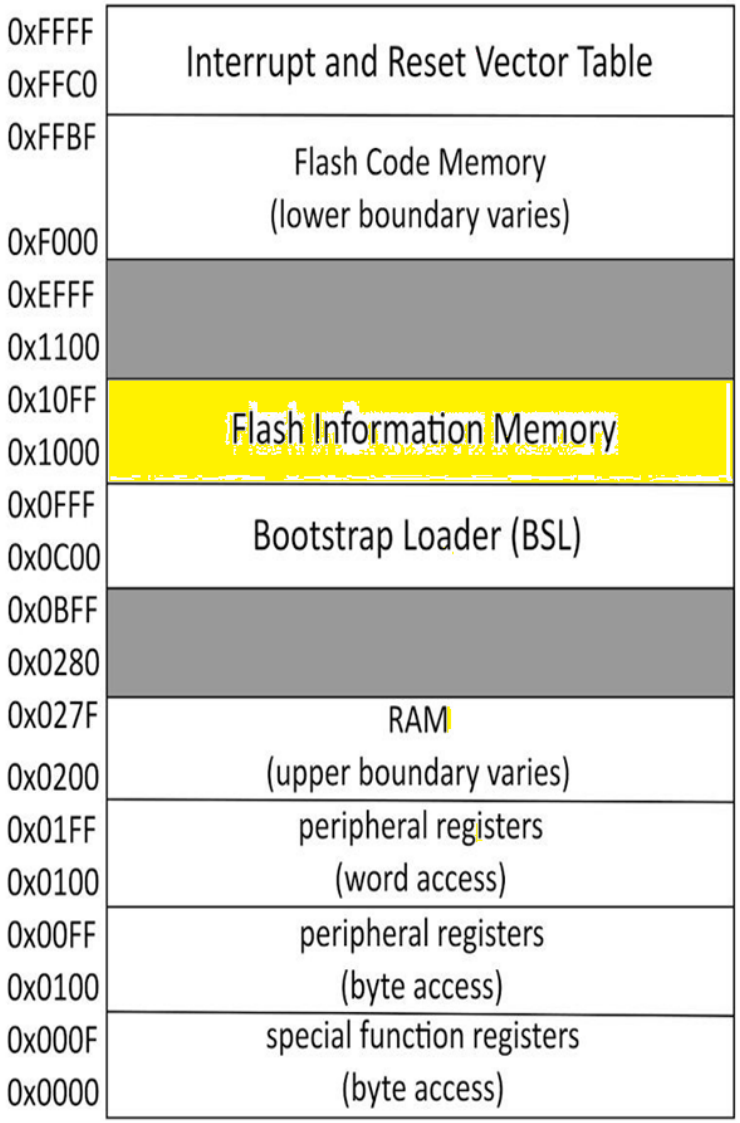
Table 5. Interrupt Sources, Flags, and Vectors

INTERRUPT SOURCE	INTERRUPT FLAG	SYSTEM INTERRUPT	WORD ADDRESS	PRIORITY
Power-Up External Reset Watchdog Timer+ Flash key violation PC out-of-range(1)	PORIFG RSTIFG WDTIFG KEYV(2)	Reset	0FFFEh	31, highest
NMI Oscillator fault Flash memory access violation	NMIIFG OFIFG ACCVIFG(2)(3)	(non)-maskable (non)-maskable (non)-maskable	0FFFCh	30
Timer1_A3	TA1CCR0 CCIFG(4)	maskable	0FFFAh	29
Timer1_A3	TA1CCR2 TA1CCR1 CCIFG, TAIFG(2)(4)	maskable	0FFF8h	28
Comparator_A+	CAIFG(4)	maskable	0FFF6h	27
Watchdog Timer+	WDTIFG	maskable	0FFF4h	26
Timer0_A3	TA0CCR0 CCIFG(4)	maskable	0FFF2h	25
Timer0_A3	TA0CCR2 TA0CCR1 CCIFG, TAIFG (5)(4)	maskable	0FFF0h	24
USCI_A0/USCI_B0 receive USCI_B0 I^2C status	UCA0RXIFG, UCB0RXIFG(2)(5)	maskable	0FFEEh	23
USCI_A0/USCI_B0 transmit USCI_B0 I^2C receive/transmit	UCA0TXIFG, UCB0TXIFG(2)(6)	maskable	0FFECh	22
ADC10 (MSP430G2x53 only)	ADC10IFG(4)	maskable	0FFEAh	21
			0FFE8h	20
I/O Port P2 (up to eight flags)	P2IFG.0 to P2IFG.7(2)(4)	maskable	0FFE6h	19
I/O Port P1 (up to eight flags)	P1IFG.0 to P1IFG.7(2)(4)	maskable	0FFE4h	18
			0FFE2h	17
			0FFE0h	16
See (7)			0FFDEh	15
See (8)			0FFDEh to 0FFC0h	14 to 0, lowest

(1) A reset is generated if the CPU tries to fetch instructions from within the module register memory address range (0h to 01FFh) or from within unused address ranges.
(2) Multiple source flags
(3) (non)-maskable: the individual interrupt-enable bit can disable an interrupt event, but the general interrupt enable cannot.
(4) Interrupt flags are located in the module.
(5) In SPI mode: UCB0RXIFG. In I^2C mode: UCALIFG, UCNACKIFG, ICSTTIFG, UCSTPIFG.
(6) In UART or SPI mode: UCB0TXIFG. In I^2C mode: UCB0RXIFG, UCB0TXIFG.
(7) This location is used as bootstrap loader security key (BSLSKEY). A 0xAA55 at this location disables the BSL completely. A zero (0h) disables the erasure of the flash if an invalid password is supplied.
(8) The interrupt vectors at addresses 0FFDEh to 0FFC0h are not used in this device and can be used for regular program code if necessary.

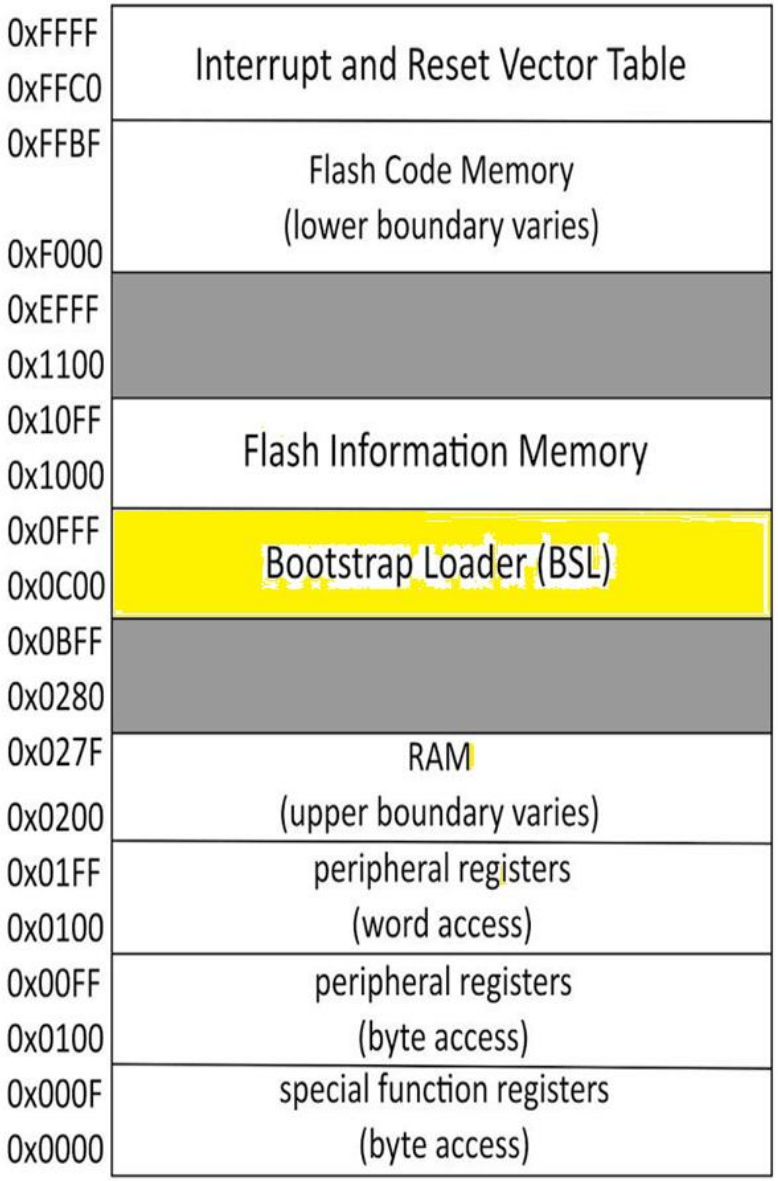
Memory Organization (Information Memory)

- Information memory is a 256B block of flash memory that is intended for storage of nonvolatile data.
- Memory address range is: 0x1000h to 0x10FFh.
- The information memory has 4 segments of 64 bytes each. Segment A contains factory calibration data for the DCO in the MSP430G2553.
- After reset, segment A is protected against programming and erasing.



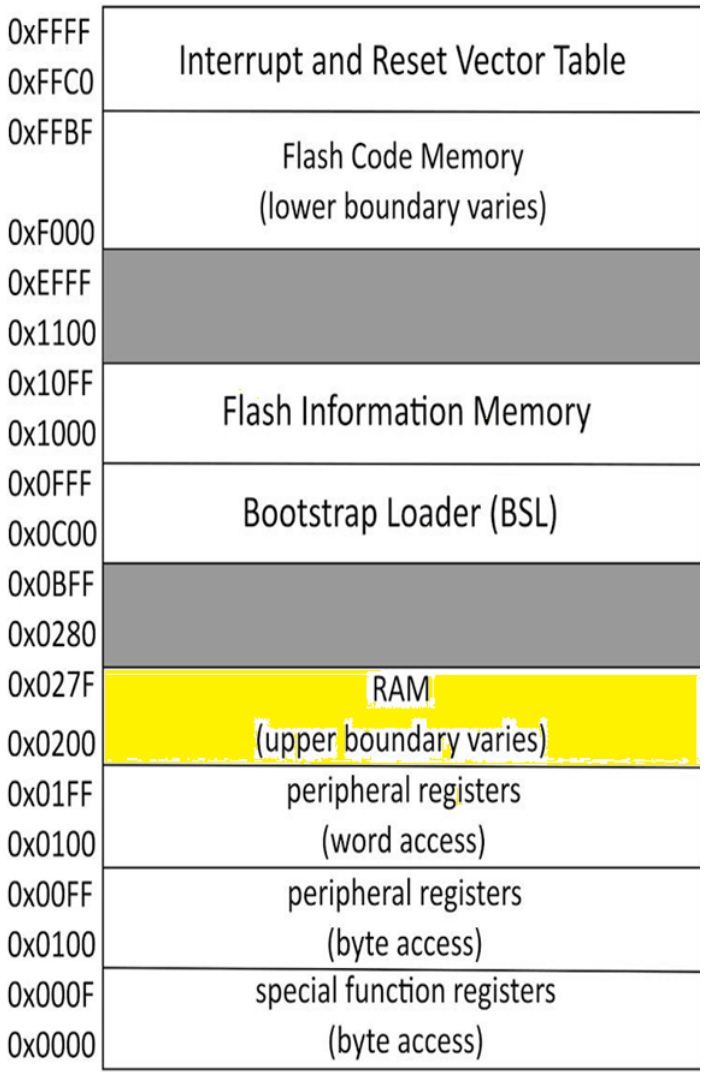
Memory Organization (Information Memory)

- Bootstrap Loader memory is a 1023B block of flash memory that is intended for storage of nonvolatile data.
- Memory address range is: 0x0C00h to 0x0FFFh.
- Bootstrap Loader (BSL) memory space contains code which is invoked when the controllers enters into BSL programming mode



Memory Organization (RAM)

- RAM is used for data/variables.
- RAM starts at 0200h. The end address of RAM depends on the amount of RAM present and varies by device.
- MSP430G2553 has 512 Bytes of RAM.



Memory Organization (Peripheral Modules)

- Peripheral registers are used by CPU to access and configure peripherals.
- The address space from 0100 to 01FFh is reserved for 16-bit peripheral modules. These modules should be accessed with word instructions. If byte instructions are used, only even addresses are permissible, and the high byte of the result is always 0.
- You can find the details of each peripheral register and its address in the ‘Peripheral File Map’ in the datasheet.

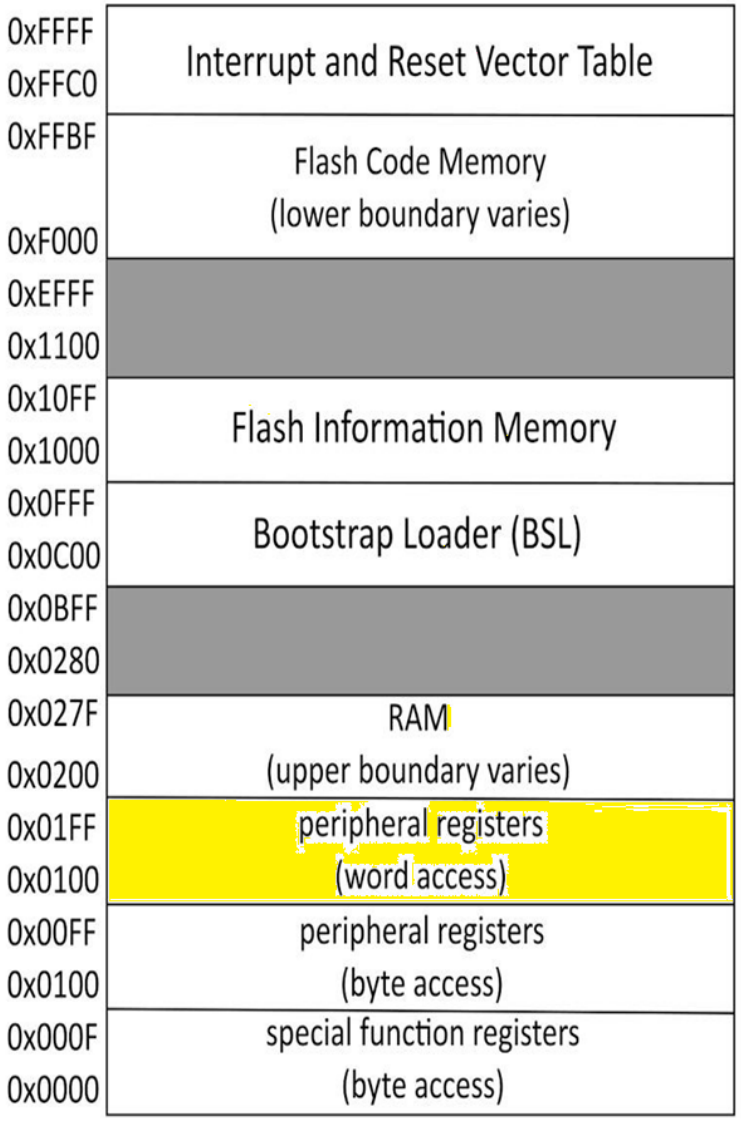


Table 14. Peripherals With Word Access

MODULE	REGISTER DESCRIPTION	REGISTER NAME	OFFSET
ADC10 (MSP430G2x53 devices only)	ADC data transfer start address	ADC10SA	1BCh
	ADC memory	ADC10MEM	1B4h
	ADC control register 1	ADC10CTL1	1B2h
	ADC control register 0	ADC10CTL0	1B0h
Timer1_A3	Capture/compare register	TA1CCR2	0196h
	Capture/compare register	TA1CCR1	0194h
	Capture/compare register	TA1CCR0	0192h
	Timer_A register	TA1R	0190h
	Capture/compare control	TA1CCTL2	0186h
	Capture/compare control	TA1CCTL1	0184h
	Capture/compare control	TA1CCTL0	0182h
	Timer_A control	TA1CTL	0180h
	Timer_A interrupt vector	TA1IV	011Eh
Timer0_A3	Capture/compare register	TA0CCR2	0176h
	Capture/compare register	TA0CCR1	0174h
	Capture/compare register	TA0CCR0	0172h
	Timer_A register	TA0R	0170h
	Capture/compare control	TA0CCTL2	0166h
	Capture/compare control	TA0CCTL1	0164h
	Capture/compare control	TA0CCTL0	0162h
	Timer_A control	TA0CTL	0160h
	Timer_A interrupt vector	TA0IV	012Eh
Flash Memory	Flash control 3	FCTL3	012Ch
	Flash control 2	FCTL2	012Ah
	Flash control 1	FCTL1	0128h
Watchdog Timer+	Watchdog/timer control	WDTCTL	0120h

Memory Organization (Peripheral Modules)

- The address space from 010h to 0FFh is reserved for 8-bit peripheral modules. These modules should be accessed with byte instructions.
- Read access of byte modules using word instructions results in unpredictable data in the high byte. If word data is written to a byte module only the low byte is written into the peripheral register, ignoring the high byte.

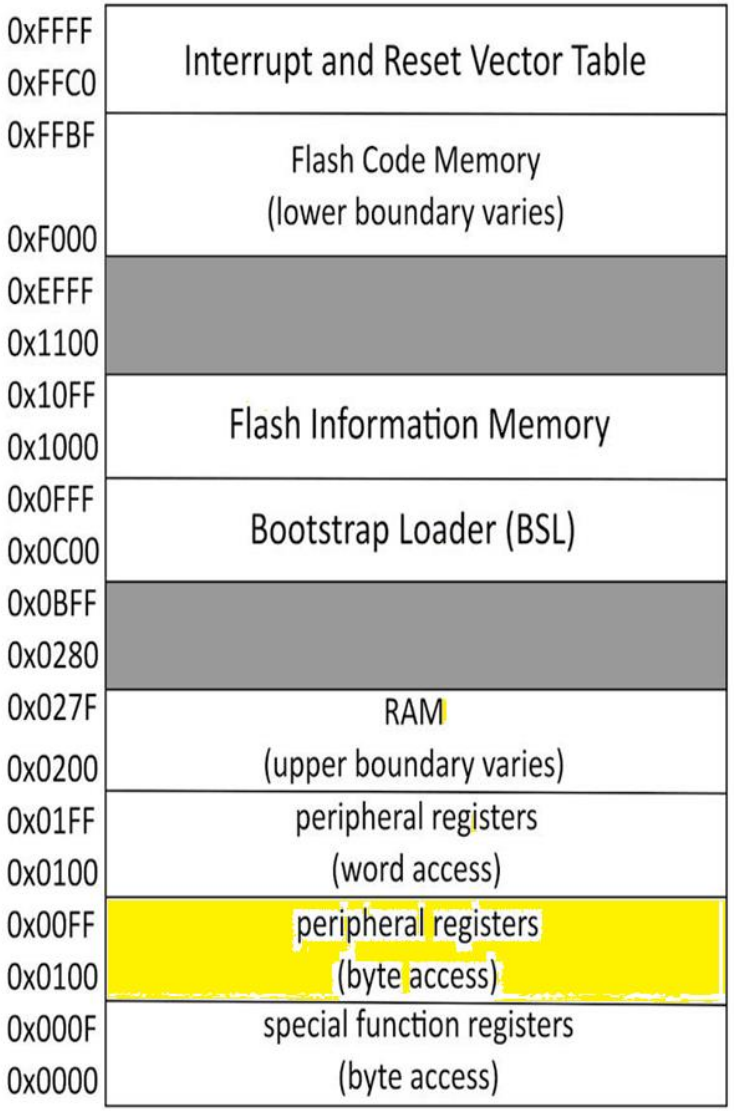


Table 15. Peripherals With Byte Access

MODULE	REGISTER DESCRIPTION	REGISTER NAME	OFFSET
USCI_B0	USCI_B0 transmit buffer	UCB0TXBUF	06Fh
	USCI_B0 receive buffer	UCB0RXBUF	06Eh
	USCI_B0 status	UCB0STAT	06Dh
	USCI B0 I2C Interrupt enable	UCB0CIE	06Ch
	USCI_B0 bit rate control 1	UCB0BR1	06Bh
	USCI_B0 bit rate control 0	UCB0BR0	06Ah
	USCI_B0 control 1	UCB0CTL1	069h
	USCI_B0 control 0	UCB0CTL0	068h
	USCI_B0 I2C slave address	UCB0SA	011Ah
	USCI_B0 I2C own address	UCB0OA	0118h
USCI_A0	USCI_A0 transmit buffer	UCA0TXBUF	067h
	USCI_A0 receive buffer	UCA0RXBUF	066h
	USCI_A0 status	UCA0STAT	065h
	USCI_A0 modulation control	UCA0MCTL	064h
	USCI_A0 baud rate control 1	UCA0BR1	063h
	USCI_A0 baud rate control 0	UCA0BR0	062h
	USCI_A0 control 1	UCA0CTL1	061h
	USCI_A0 control 0	UCA0CTL0	060h
	USCI_A0 IrDA receive control	UCA0IRRCTL	05Fh
	USCI_A0 IrDA transmit control	UCA0IRTCTL	05Eh
	USCI_A0 auto baud rate control	UCA0ABCTL	05Dh
ADC10 (MSP430G2x53 devices only)	ADC analog enable 0	ADC10AE0	04Ah
	ADC analog enable 1	ADC10AE1	04Bh
	ADC data transfer control register 1	ADC10DTC1	049h
	ADC data transfer control register 0	ADC10DTC0	048h
Comparator_A+	Comparator_A+ port disable	CAPD	05Bh
	Comparator_A+ control 2	CACTL2	05Ah
	Comparator_A+ control 1	CACTL1	059h
Basic Clock System+	Basic clock system control 3	BCSCTL3	053h
	Basic clock system control 2	BCSCTL2	058h
	Basic clock system control 1	BCSCTL1	057h
	DCO clock frequency control	DCOCTL	056h
Port P3 (28-pin PW and 32-pin RHB only)	Port P3 selection 2. pin	P3SEL2	043h
	Port P3 resistor enable	P3REN	010h
	Port P3 selection	P3SEL	01Bh
	Port P3 direction	P3DIR	01Ah
	Port P3 output	P3OUT	019h
	Port P3 input	P3IN	018h
Port P2	Port P2 selection 2	P2SEL2	042h
	Port P2 resistor enable	P2REN	02Fh
	Port P2 selection	P2SEL	02Eh
	Port P2 interrupt enable	P2IE	02Dh
	Port P2 interrupt edge select	P2IES	02Ch
	Port P2 interrupt flag	P2IFG	02Bh
	Port P2 direction	P2DIR	02Ah
	Port P2 output	P2OUT	029h
	Port P2 input	P2IN	028h

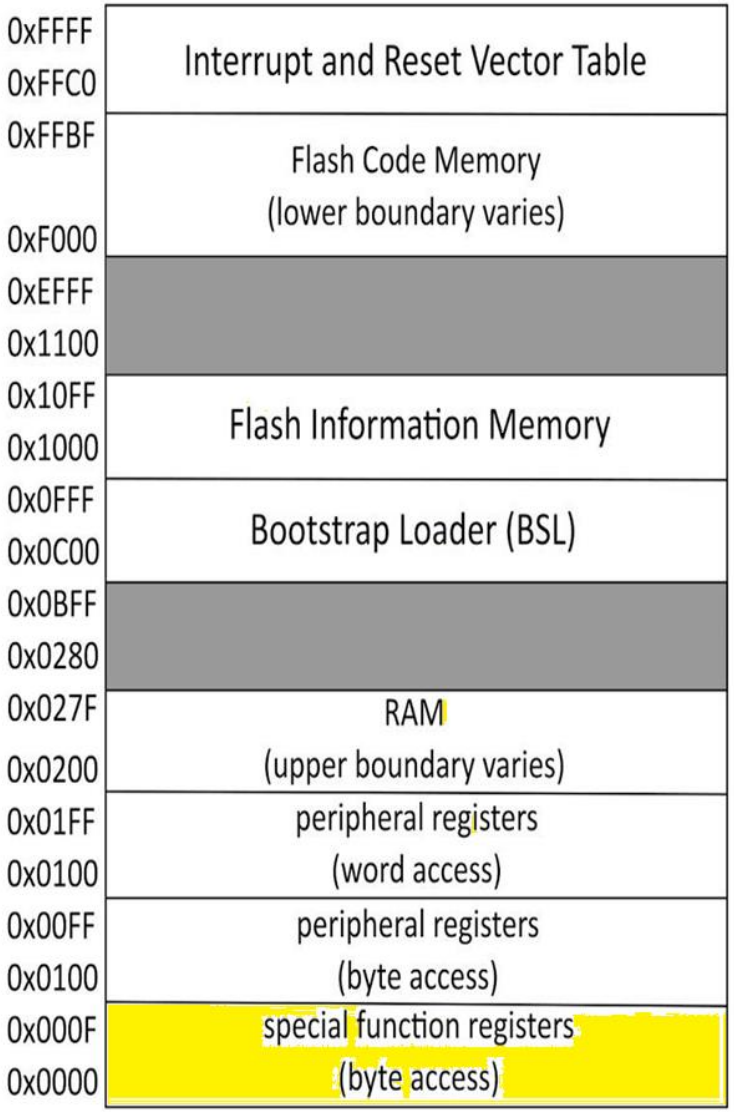
Table 15. Peripherals With Byte Access (continued)

MODULE	REGISTER DESCRIPTION	REGISTER NAME	OFFSET
Port P1	Port P1 selection 2	P1SEL2	041h
	Port P1 resistor enable	P1REN	027h
	Port P1 selection	P1SEL	026h
	Port P1 interrupt enable	P1IE	025h
	Port P1 interrupt edge select	P1IES	024h
	Port P1 interrupt flag	P1IFG	023h
	Port P1 direction	P1DIR	022h
	Port P1 output	P1OUT	021h
	Port P1 input	P1IN	020h
Special Function	SFR interrupt flag 2	IFG2	003h
	SFR interrupt flag 1	IFG1	002h
	SFR interrupt enable 2	IE2	001h
	SFR interrupt enable 1	IE1	000h

Memory Organization (Special Function Registers)

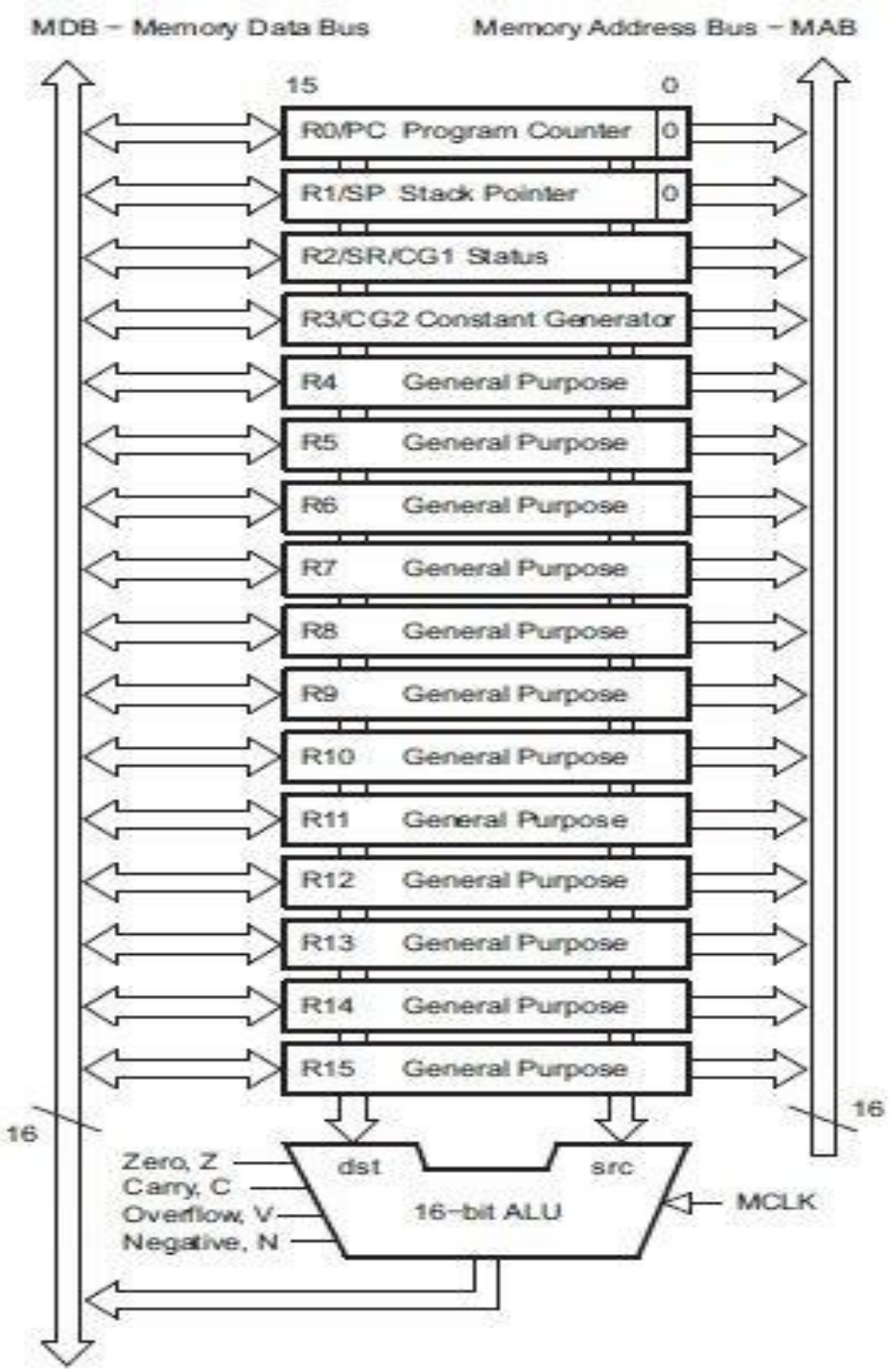
- Some peripheral functions are configured in the SFRs. The SFRs are located in the lower 16 bytes of the address space, and are organized by byte. SFRs must be accessed using byte instructions only.

- MSP430G2553 has interrupt enables and interrupt flag registers as SFRs.



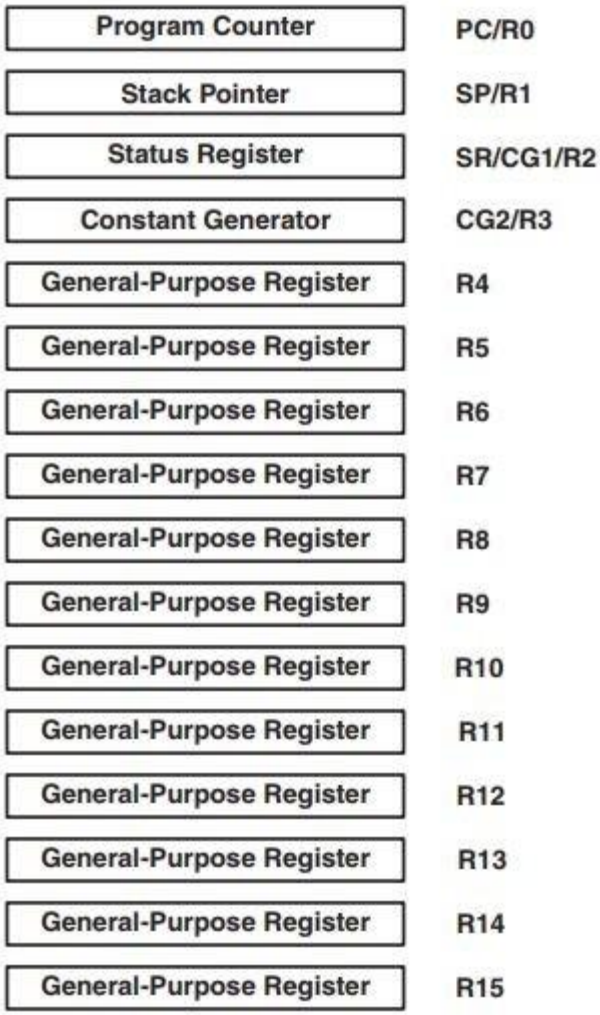

PROGRESSIVE EDUCATION SOCIETY
Pune - 5
Modern College of Engineering
Pune - 5

MSP430 CPU


MDB – Memory Data Bus
Memory Address Bus – MAB
15
0
R0/PC Program Counter
0
R1/SP Stack Pointer
0
R2/SR/CG1 Status
R3/CG2 Constant Generator
R4 General Purpose
R5 General Purpose
R6 General Purpose
R7 General Purpose
R8 General Purpose
R9 General Purpose
R10 General Purpose
R11 General Purpose
R12 General Purpose
R13 General Purpose
R14 General Purpose
R15 General Purpose
16
16
Zero, Z
Carry, C
Overflow, V
Negative, N
dst
src
16-bit ALU
MCLK

CPU Registers

- 16 Registers: The generous set of 16 registers is characteristic of a RISC CPU.
- These Registers provide reduced instruction execution time.
- Register to register operation takes only clock cycle.
- 4 Special Purpose
  - Program Counter(PC)
  - Stack Pointer(SP)
  - Status Register(SR)
  - Constant Generator
- 12 General Purpose

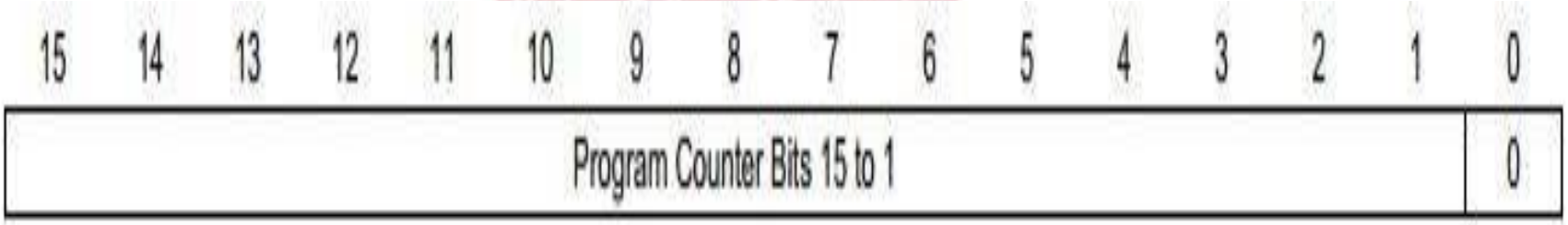


CPU Registers

Special CPU Register

Program Counter

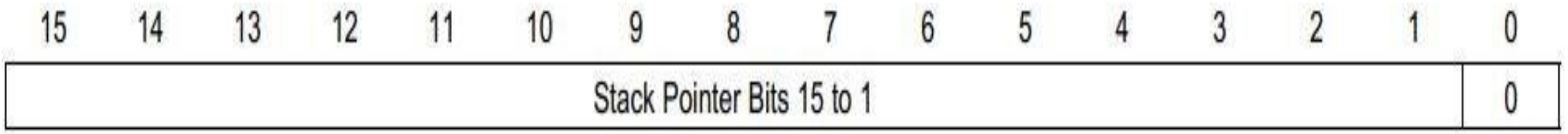
- This contains the address of the next instruction to be executed. Instructions are composed of 1–3 words, which must be aligned to even addresses, so the LSB of the PC is hard-wired to 0.



Special CPU Register

Stack Pointer

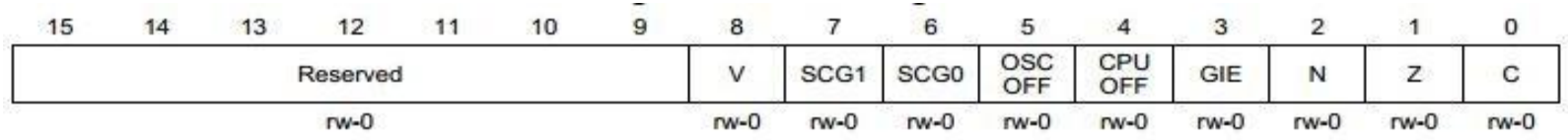
- The stack pointer is used by the CPU to store the return addresses of subroutine calls and interrupts.
- The SP is initialized into RAM by the user, and is aligned to even addresses.
- SP hold the address of the most recently added word in the stack and is automatically adjusted as the stack goes up and down.



Special CPU Register

Status Register

- This contains a set of flags (single bits), whose functions fall into three categories.
  - The most commonly used flags are C, Z, N, and V, which give information about the result of the last arithmetic or logical operation. Decisions that affect the flow of control in the program can be made by testing these bits.
  - Setting the GIE bit enables maskable interrupts.
  - The final group of bits is CPUOFF, OSCOFF, SCG0, and SCG1, which control the mode of operation of the MCU.



Special CPU Register

Status Register

Bit	Description
V	Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range.
	`ADD(.B),ADDC(.B)` Set when: Positive + Positive = Negative Negative + Negative = Positive Otherwise reset
	`SUB(.B),SUBC(.B),CMP(.B)` Set when: Positive – Negative = Negative Negative – Positive = Positive Otherwise reset
SCG1	System clock generator 1. When set, turns off the SMCLK.
SCG0	System clock generator 0. When set, turns off the DCO dc generator, if DCOCLK is not used for MCLK or SMCLK.
OSCOFF	Oscillator Off. When set, turns off the LFXT1 crystal oscillator, when LFXT1CLK is not use for MCLK or SMCLK.
CPUOFF	CPU off. When set, turns off the CPU.
GIE	General interrupt enable. When set, enables maskable interrupts. When reset, all maskable interrupts are disabled.
N	Negative bit. Set when the result of a byte or word operation is negative and cleared when the result is not negative. Word operation: N is set to the value of bit 15 of the result. Byte operation: N is set to the value of bit 7 of the result.
Z	Zero bit. Set when the result of a byte or word operation is 0 and cleared when the result is not 0.
C	Carry bit. Set when the result of a byte or word operation produced a carry and cleared when no carry occurred.

Special CPU Register

Constant Generator Registers

- Six commonly-used constants are generated with the constant generator registers R2 and R3, without requiring an additional 16-bit word of program code. The constants are selected with the source-register addressing modes (As).
- The constant generator advantages are:
  - No special instructions required
  - No additional code word for the six constants
  - No code memory access required to fetch the constant

CPU Registers

General Purpose Registers

- The remaining 12 registers R4–R15 have no dedicated purpose and may be used as general working registers.
- They may be used for either data or addresses because both are 16-bit values, which simplifies the operation significantly.

MSP430 Instruction Set Architecture

Instruction Format

There are three formats of instructions in the MSP430:

- Double operand: Arithmetic and logical operations with two operands such as ADD R4,R5. Both operands must be specified in the instruction. This contrasts with accumulator-based architectures, where an accumulator or working register is used automatically as the destination and one operand.
- Single operand: A mixture of instructions for control or to manipulate a single operand, which is effectively the source for the addressing modes.
- Jumps: The jump to the destination rather than its absolute address, in other words the offset that must be added to the program counter.

Table 3. Instruction Word Formats

INSTRUCTION FORMAT	EXAMPLE	OPERATION
Dual operands, source-destination	ADD R4,R5	R4 + R5 ---> R5
Single operands, destination only	CALL R8	PC -->(TOS), R8--> PC
Relative jump, un/conditional	JNE	Jump-on-equal bit = 0

Addressing Modes

- Addressing modes are the ways in which operands can be specified.
- MSP430 has seven addressing modes. These are:
  1. Register Mode
  2. Indexed Mode
  3. Symbolic Mode
  4. Absolute Mode
  5. Indirect Register Mode
  6. Indirect Autoincrement Mode
  7. Immediate Mode

Addressing Modes

- Register mode: The operand is in one of the CPU's registers so there are no problems with the address. The most significant bits of the destination are cleared if the operation produces 8-bit or 16-bit data.
- Indirect register mode: One of the CPU's registers contains the address of the operand, which is always treated as a 20-bit value in the MSP430X.
- Indirect auto increment register mode: Again a 20-bit address is taken from one of the CPU's registers. The address is automatically incremented by 1 for a byte, 2 for a word, or 4 for an address word. There is one special case: immediate data is implemented as auto increment addressing on the program counter.

Addressing Modes

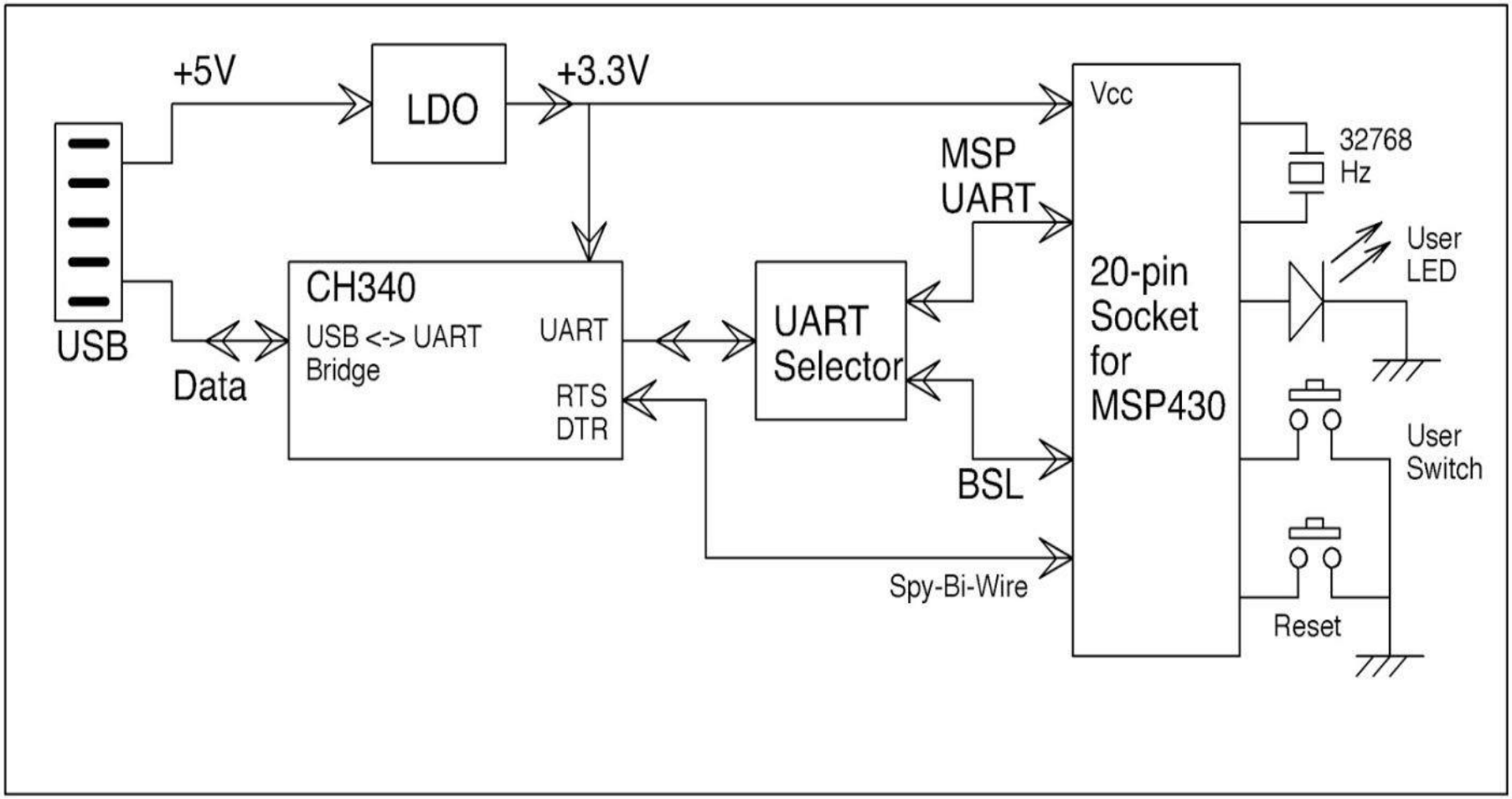
- Indexed mode: The address is given by adding the contents of a register to a constant, which follows the instruction in the program memory. The constant is a 16-bit value in the MSP430 but may have 20 bits in the MSP430X, which requires a distinct instruction. There are three special cases of indexed mode:
  - The Symbolic mode is indexed on the program counter (PC-relative).
  - The Absolute mode is indexed on the status register, which behaves as though it contains 0.
  - The SP-relative mode is indexed on the stack pointer and is not considered a separate mode by TI.

Source/Destination Operand Addressing Modes

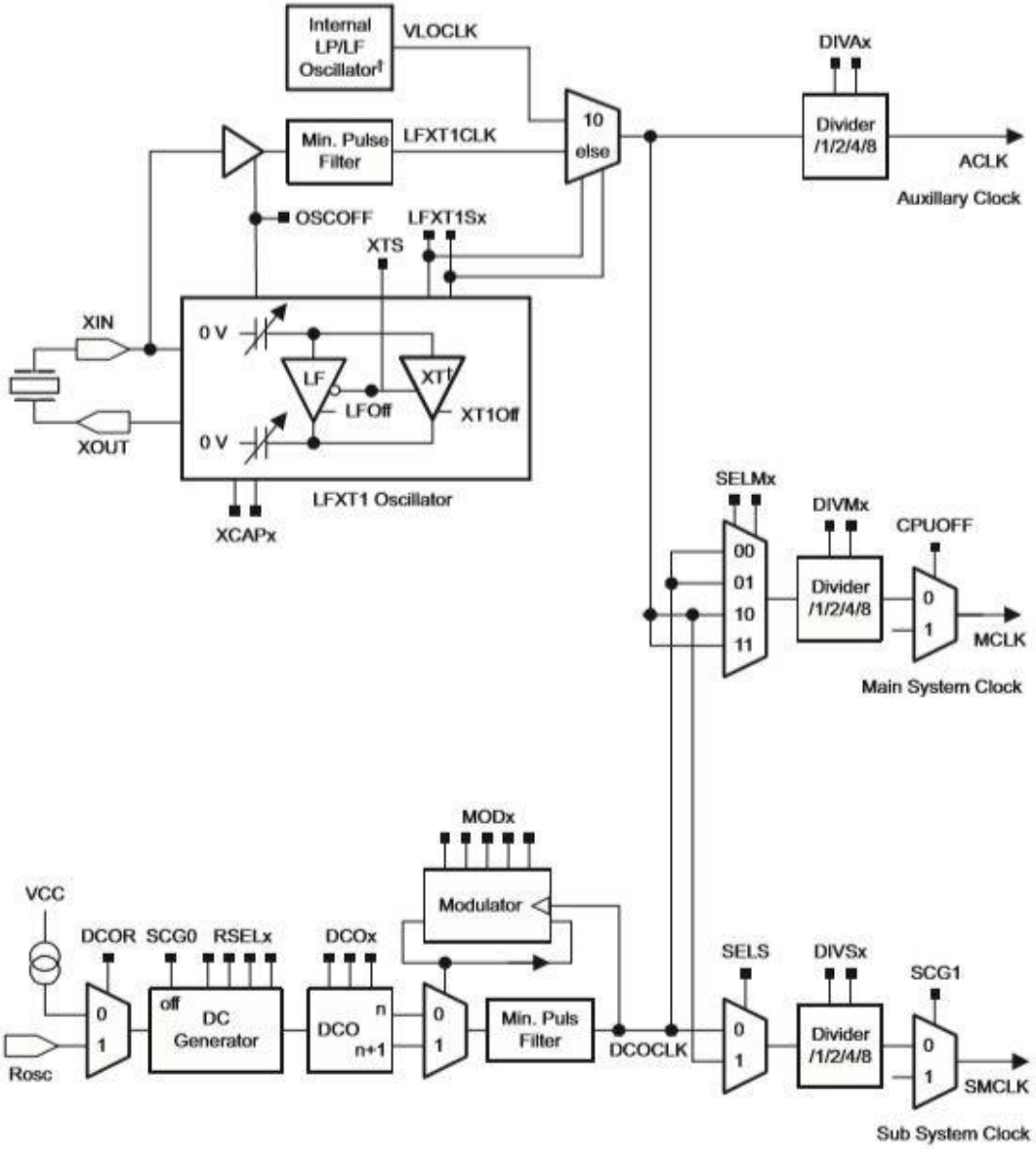
As/Ad	Addressing Mode	Syntax	Description
00/0	Register mode	Rn	Register contents are operand
01/1	Indexed mode	X(Rn)	(Rn + X) points to the operand. X is stored in the next word.
01/1	Symbolic mode	ADDR	(PC + X) points to the operand. X is stored in the next word. Indexed mode X(PC) is used.
01/1	Absolute mode	&ADDR	The word following the instruction contains the absolute address. X is stored in the next word. Indexed mode X(SR) is used.
10/-	Indirect register mode	@Rn	Rn is used as a pointer to the operand.
11/-	Indirect autoincrement	@Rn+	Rn is used as a pointer to the operand. Rn is incremented afterwards by 1 for .B instructions and by 2 for .W instructions.
11/-	Immediate mode	#N	The word following the instruction contains the immediate constant N. Indirect autoincrement mode @PC+ is used.

Typical MSP430 development board

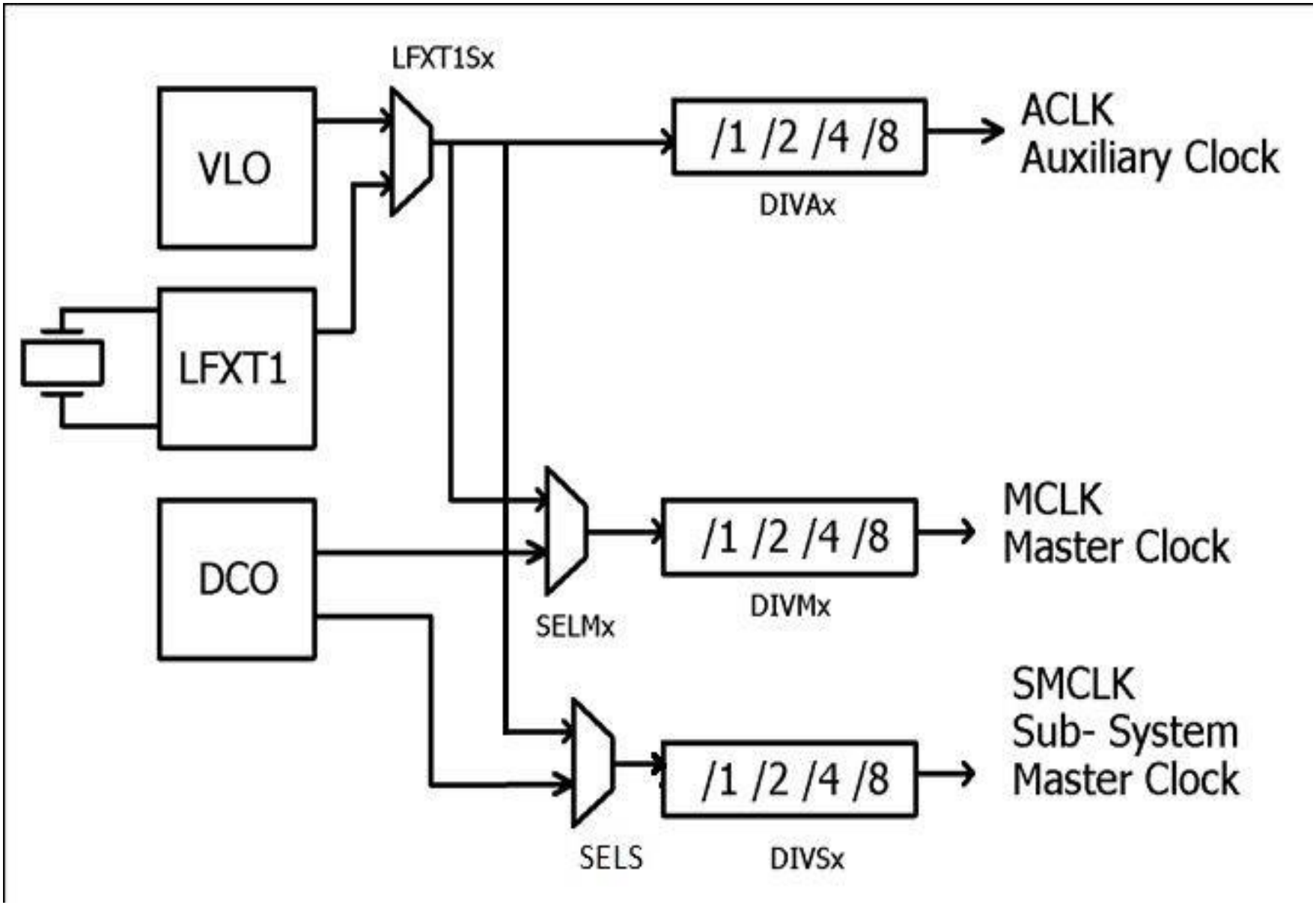
- MSP430 development board is a low cost DIY platform for learning microcontrollers and physical computing.



Basic Clock Module



Simplified Block Diagram of MSP430G2553 Clock System



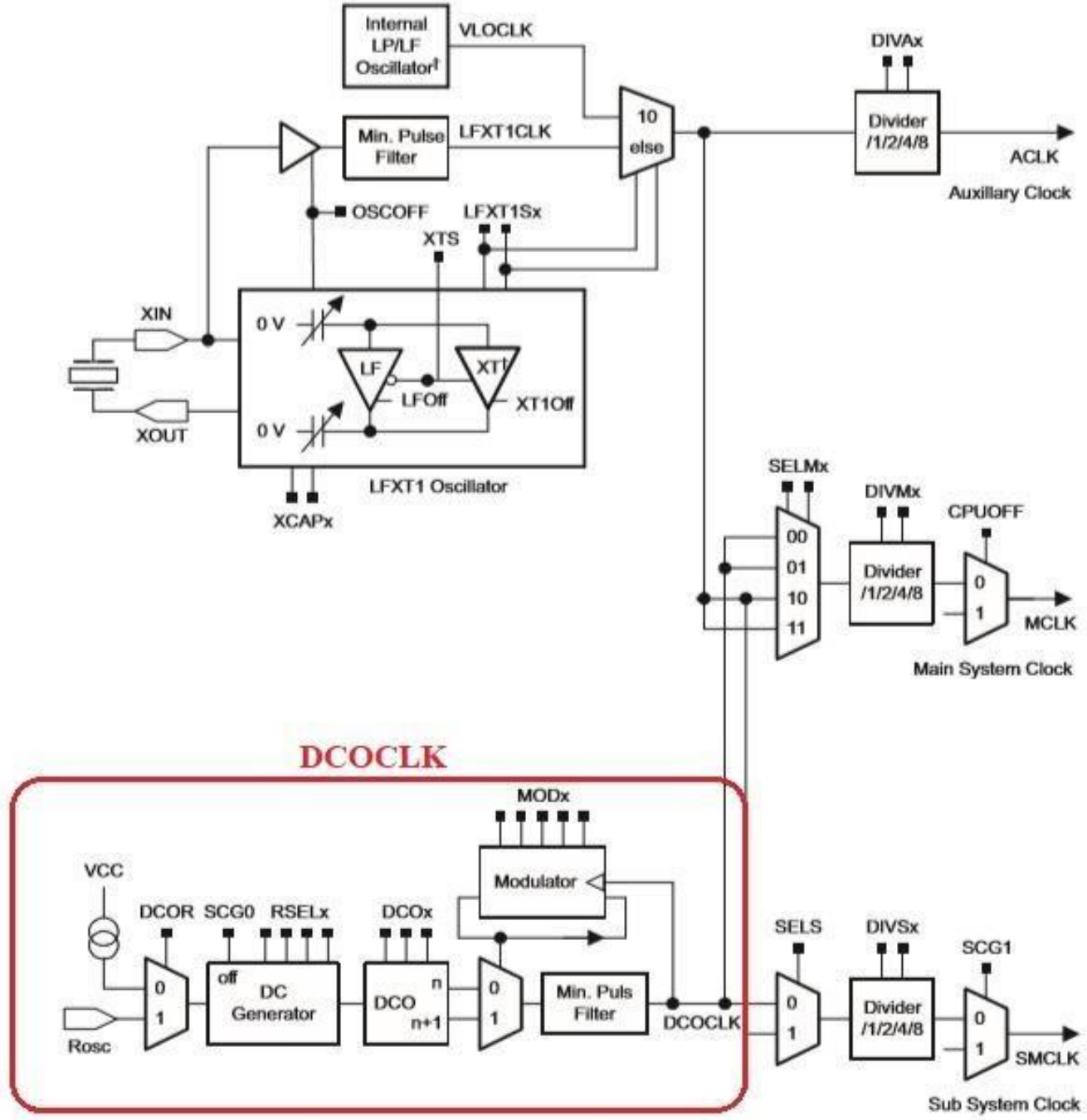
Clock Sources

- MSP430 can get clock from 3 sources:
  1. DCO
  2. LFXT1
  3. VLO

Clock Sources

1. **DCO (**Digitally Controlled Oscillator)

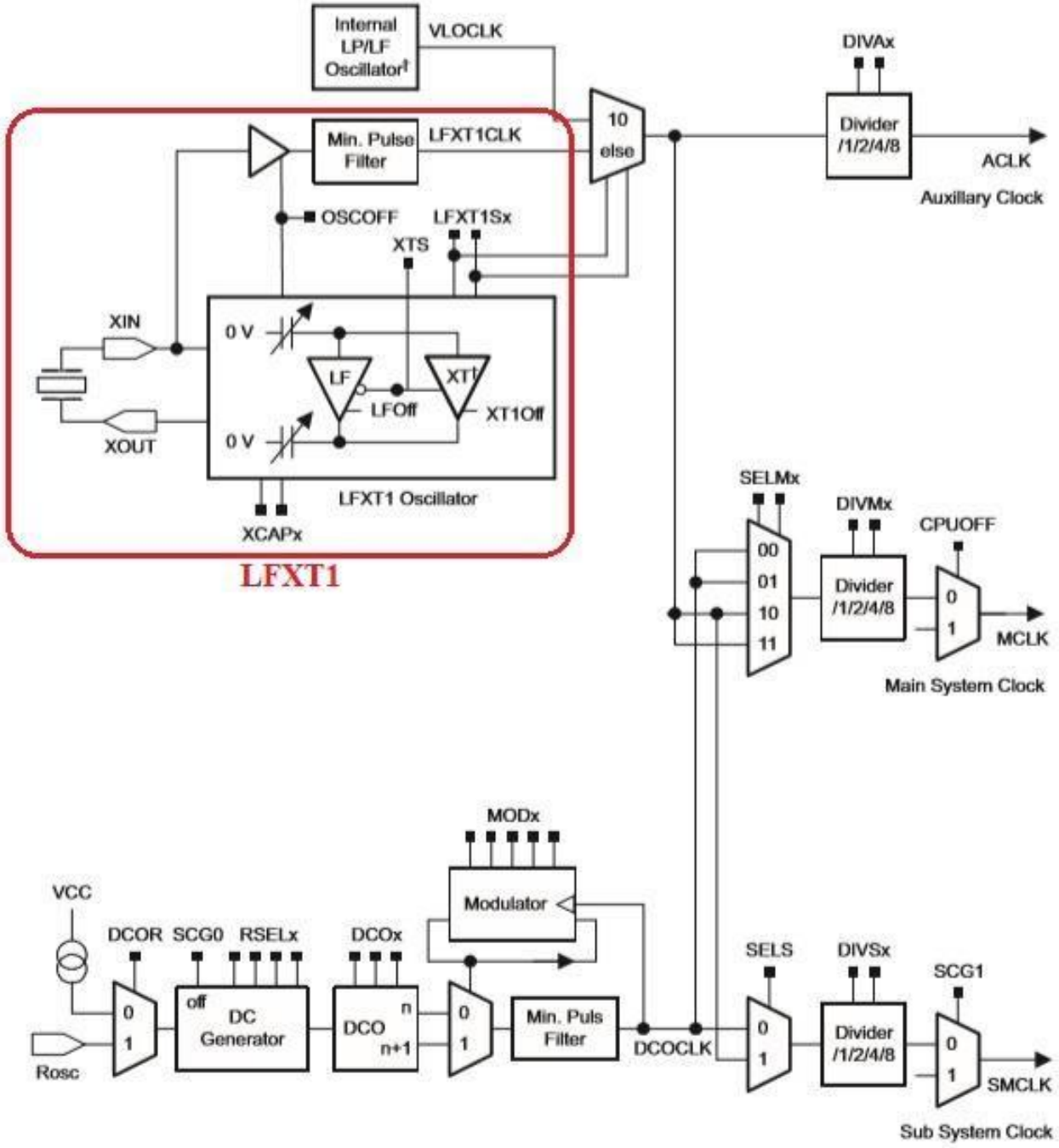
- One of the highlights of MSP430, it is basically a highly controllable RC oscillator that starts in less than 1μs. Therefore starts rapidly at full speed from Low power mode.
- The DCO frequency can be adjusted by software.
- DCO frequency ranges from 60K to 16MHz.
- Default frequency => 1.1MHz


Internal LP/LF Oscillator
VLOCLK
DIVAx
Min. Pulse Filter
LFXT1CLK
10
else
Divider /1/2/4/8
ACLK
Auxiliary Clock
OSCOFF
LFXT1Sx
XTS
XIN
0 V
LF
XT1
LFOff
XT1Off
XOUT
0 V
LFXT1 Oscillator
XCAPx
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
DCOCLK
MODx
Modulator
VCC
DCOR
SCG0
RSELx
DCOx
off
DC Generator
DCO
n
n+1
Min. Puls Filter
DCOCLK
SELS
DIVSx
SCG1
Rosc
Divider /1/2/4/8
SMCLK
Sub System Clock

Clock Sources

2. LFXT1 (Low or high frequency crystal oscillator)

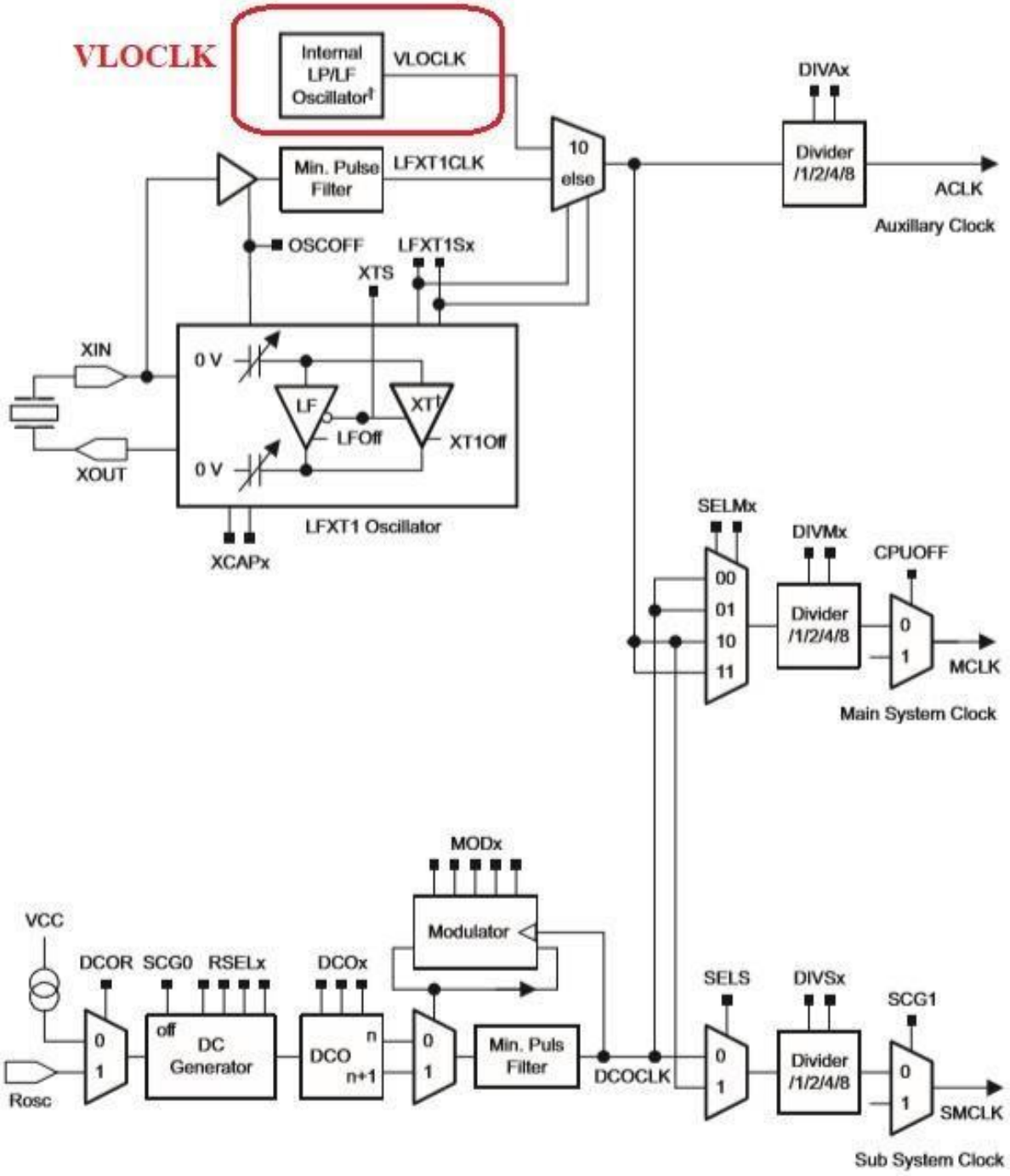
- Used with low frequency watch crystal (32 kHz)
- Also used with a high frequency crystal (400 kHz to 16MHz) (Absent in MSP430G2553)


Internal LP/LF Oscillator†
VLOCLK
Min. Pulse Filter
LFXT1CLK
10 else
DIVAx
Divider /1/2/4/8
ACLK
Auxiliary Clock
OSCOFF
LFXT1Sx
XTS
XIN
XOUT
0 V
0 V
LF
XT1†
LFOff
XT1Off
LFXT1 Oscillator
XCAPx
LFXT1
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
MODx
Modulator
VCC
DCOR
SCG0
RSELx
DCOx
off
DC Generator
DCO
n
n+1
Min. Puls Filter
DCOCLK
SELS
DIVSx
SCG1
Divider /1/2/4/8
SMCLK
Rosc
Sub System Clock

Clock Sources

3. VLO

- Internal very low-power, low-frequency oscillator
- Typical frequency 12kHz


VLOCLK
Internal LP/LF Oscillator†
VLOCLK
DIVAx
Divider /1/2/4/8
ACLK
Auxiliary Clock
10
else
Min. Pulse Filter
LFXT1CLK
OSCOFF
LFXT1Sx
XTS
XIN
XOUT
0 V
0 V
LF
XT†
LFOff
XT1Off
LFXT1 Oscillator
XCAPx
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
MODx
Modulator
VCC
DCOR
SCG0
RSELx
DCOx
SELS
DIVSx
SCG1
0
1
off
DC Generator
DCO
n
n+1
Min. Puls Filter
DCOCLK
Divider /1/2/4/8
SMCLK
Rosc
Sub System Clock

Clock Signals

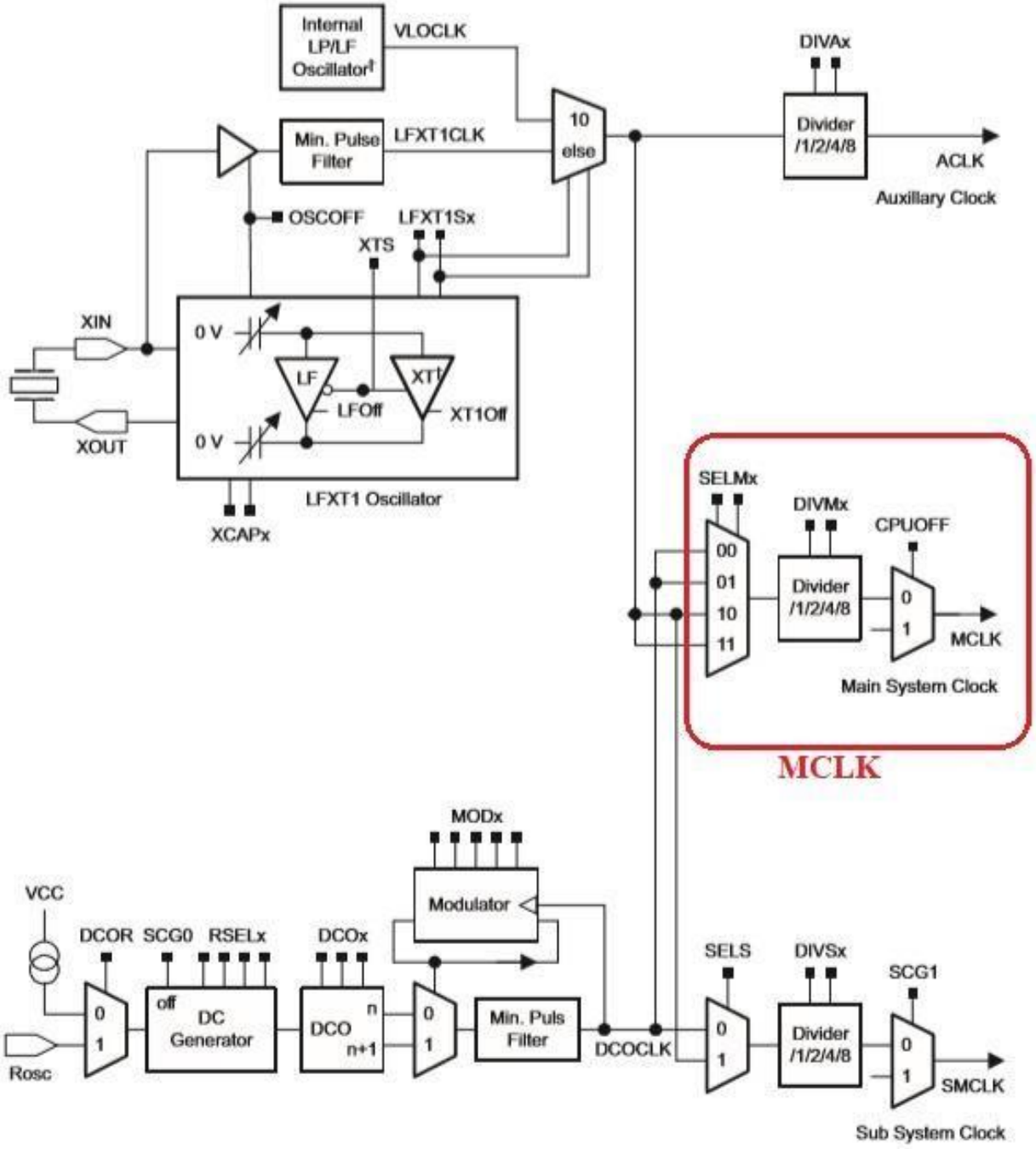
- Why do we need so many different sources of clock? Because of conflicting demands of high performance and low power.
- We need different kinds of clock for different purposes:
  - Low clock frequency for energy conservation and time keeping
  - High clock frequency for fast reaction to events and fast burst processing capability
  - Clock stability over operating temperature and supply voltage

- Using three internal clock signals, the user can select the best balance of performance and low power consumption.

Clock Signals

1. MCLK: Master clock

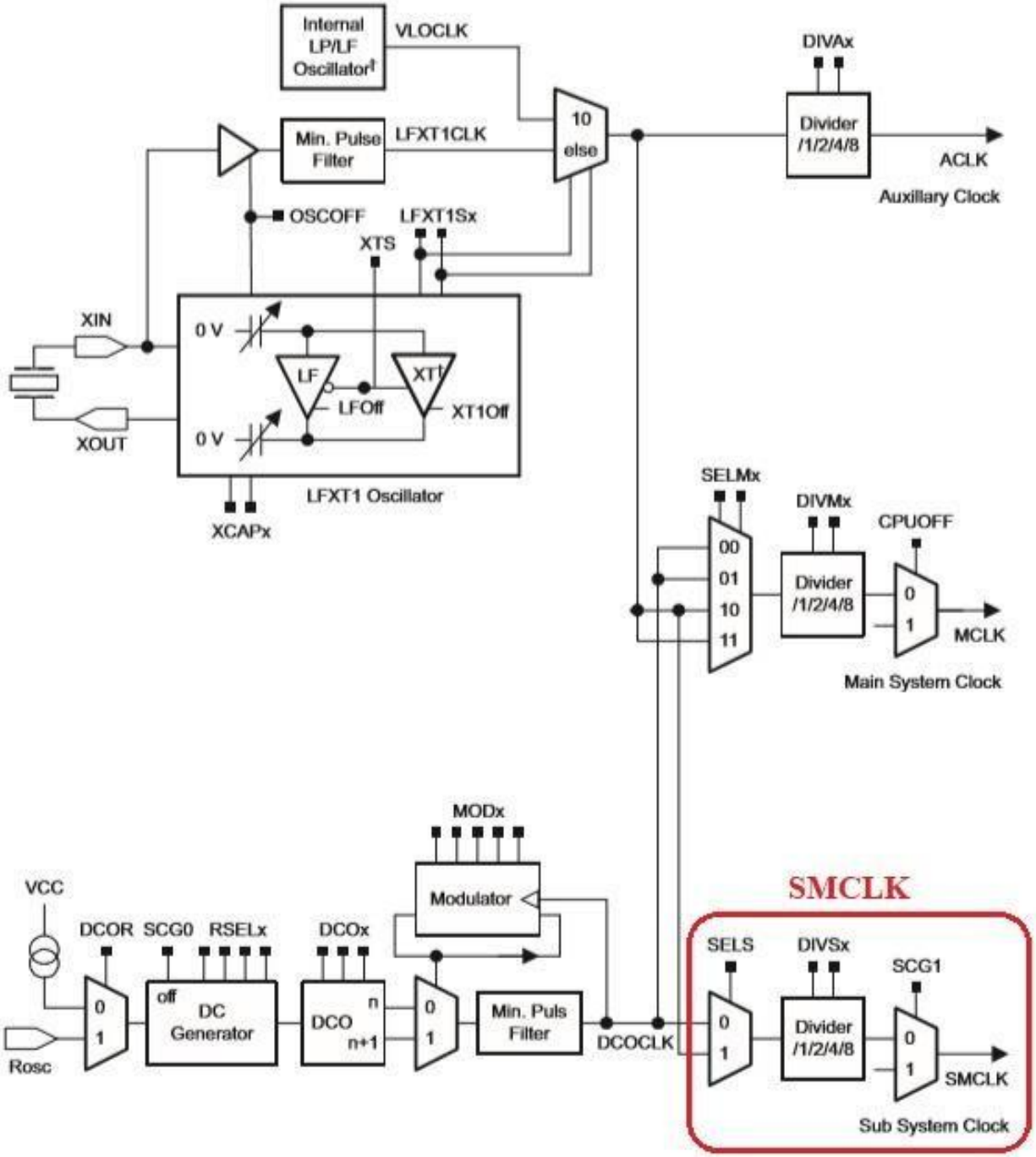
- Used by the CPU and a few peripherals.
- Supplied by the DCO with a frequency of around 1.1MHz. Stabilized by PLL.
- MCLK is software selectable as LFXT1CLK, VLOCLK, DCOCLK (or XT2CLK, but not present on MSP430G2553).


Internal LP/LF Oscillator†
VLOCLK
DIVAx
Min. Pulse Filter
LFXT1CLK
10
else
Divider /1/2/4/8
ACLK
Auxiliary Clock
OSCOFF
LFXT1Sx
XTS
XIN
0 V
LF
XT1
LFOff
XT1Off
XOUT
0 V
LFXT1 Oscillator
XCAPx
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
MCLK
MODx
Modulator
VCC
DCOR
SCG0
RSELx
DCOx
SELS
DIVSx
SCG1
0
1
off
DC Generator
DCO
n
n+1
Min. Puls Filter
DCOCLK
Divider /1/2/4/8
SMCLK
Rosc
Sub System Clock

Clock Signals

2. SMCLK: Sub-system Master Clock

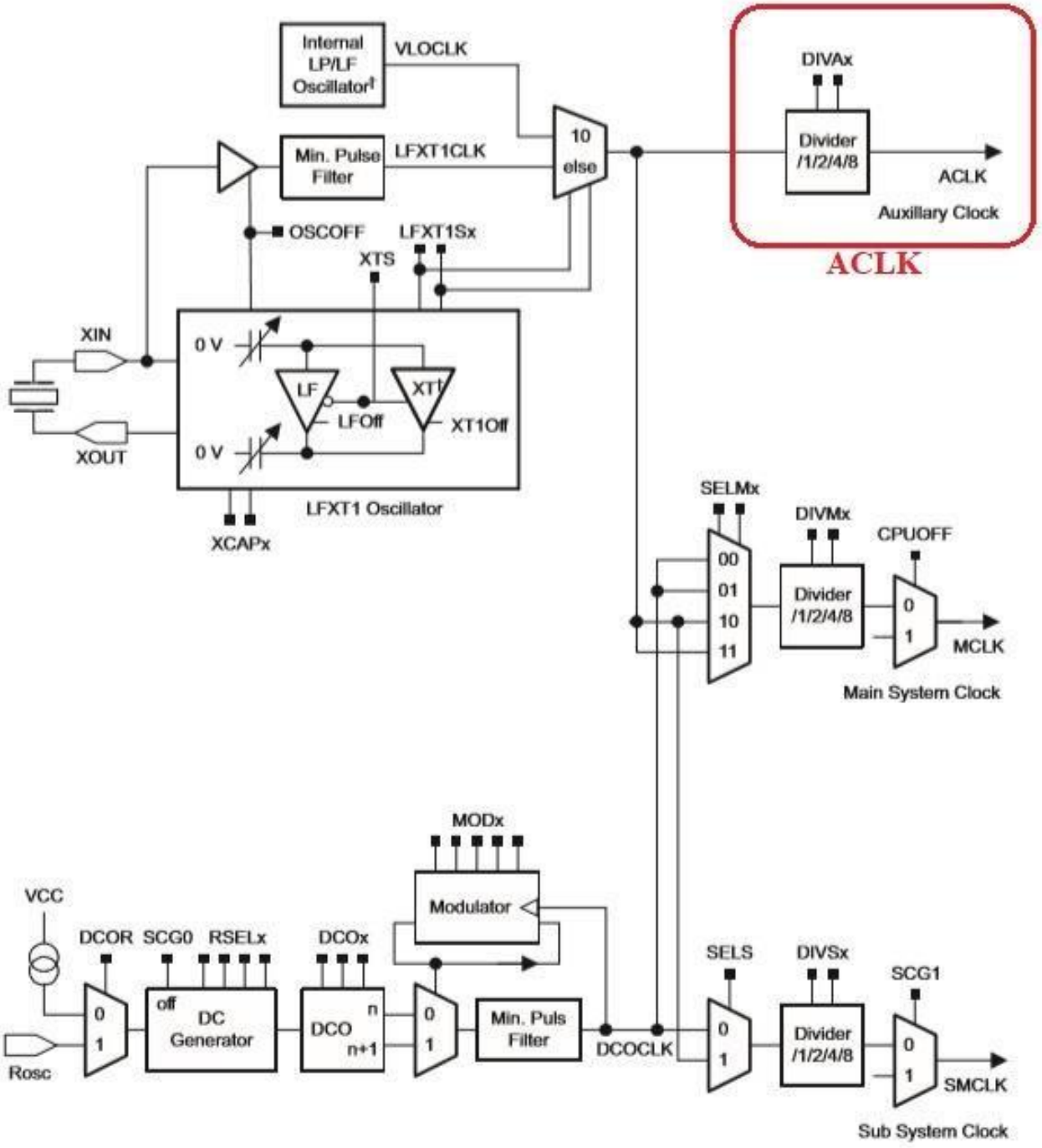
- Distributed to peripherals.
- Often same as MCLK.
- Supplied by the DCO with a frequency of around 1.1MHz. Stabilized by PLL.
- SMCLK is software selectable as LFXT1CLK, VLOCLK, DCOCLK (or XT2CLK, but not present on MSP430G2553).


Internal LP/LF Oscillator
VLOCLK
Min. Pulse Filter
LFXT1CLK
10
else
DIVAx
Divider /1/2/4/8
ACLK
Auxiliary Clock
OSCOFF
LFXT1Sx
XTS
XIN
XOUT
0 V
0 V
LF
XT1
LFOff
XT1Off
LFXT1 Oscillator
XCAPx
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
MODx
Modulator
SMCLK
VCC
DCOR
SCG0
RSELx
DCOx
SELS
DIVSx
SCG1
off
DC Generator
DCO
n
n+1
Min. Puls Filter
DCOCLK
Divider /1/2/4/8
SMCLK
Rosc
Sub System Clock

Clock Signals

3. ACLK: Auxiliary Clock

- Distributed among peripherals.
- Sourced from LFXT1CLK or VLOCLK.
- Typically much slower and usually <=32kHz.
- ACLK is software selectable as LFXT1CLK or VLOCLK


Internal LP/LF Oscillator†
VLOCLK
Min. Pulse Filter
LFXT1CLK
10
else
DIVAx
Divider /1/2/4/8
ACLK
Auxiliary Clock
ACLK
OSCOFF
LFXT1Sx
XTS
XIN
XOUT
0 V
0 V
LF
XT†
LFOff
XT1Off
LFXT1 Oscillator
XCAPx
SELMx
DIVMx
CPUOFF
00
01
10
11
Divider /1/2/4/8
0
1
MCLK
Main System Clock
MODx
Modulator
VCC
DCOR
SCG0
RSELx
DCOx
off
DC Generator
DCO
n
n+1
0
1
Min. Puls Filter
DCOCLK
SELS
DIVSx
SCG1
Divider /1/2/4/8
SMCLK
Rosc
Sub System Clock

Calibrated Frequencies of DCO

4 Calibrated frequencies:

- 1 MHz
- 8 MHz
- 12 MHz
- 16 MHz

Sample CCS Statements:

BCSCTL1 = CALBC1_1MHZ; (For range selection)

DCOCTL = CALDCO_1MHZ; (For Freq. selection)

Crystal oscillator

- Crystals are used when an accurate, stable frequency is needed.
- Crystals are cut from carefully grown, high-quality quartz with specific orientations to give them high stability.
- Traditional crystals oscillate at frequencies of a few MHz.
- Machined into complicated tuning fork shapes to give the low frequency.
- Designed to be most stable at 25 C.
- Crystal connected between XIN & XOUT.

Crystal oscillator

Disadvantage:

- Frequency is more sensitive to temperature.

SOLUTION => EXTERNAL OSCILLATOR

- Requires a **capacitance** to GND from each pin. Value depends on the crystal and is typically 10pF or 22pF.
- Part of it contributed by stray capacitance from PCB track, therefore kept short.
- External capacitance needed for many controllers/RTC ICs but integrated in MSP430.

Clock Registers

- **DCOCTL (DCO Control Register)**
- **BCSCTL1 (Basic Clock System Control Register 1)**
- **BCSCTL2 (Basic Clock System Control Register 2):** For MCLK & SMCLK manipulation
- **BCSCTL3 (Basic Clock System Control Register 3)**: For external crystal and capacitor selections

DCOCTL: DCO Control Register

7	6	5	4	3	2	1	0
DCOx			MODx				
rw-0	rw-1	rw-1	rw-0	rw-0	rw-0	rw-0	rw-0

DCOx	Bits 7-5	DCO frequency select. These bits select which of the eight discrete DCO frequencies within the range defined by the RSELx setting is selected.
MODx	Bits 4-0	Modulator selection. These bits define how often the f_{DCO+1} frequency is used within a period of 32 DCOCLK cycles. During the remaining clock cycles (32-MOD) the f_{DCO} frequency is used. Not useable when DCOx = 7.

BCSCTL1: Basic Clock Control Register 1

7	6	5	4	3	2	1	0
XT2OFF	XTS[1][2]	DIVAx		RSELx			
rw-(1)	rw-(0)	rw-(0)	rw-(0)	rw-0	rw-1	rw-1	rw-1

XT2OFF	Bit 7	XT2 off. This bit turns off the XT2 oscillator 0 XT2 is on 1 XT2 is off if it is not used for MCLK or SMCLK.
XTS	Bit 6	LFXT1 mode select. 0 Low-frequency mode 1 High-frequency mode
DIVAx	Bits 5-4	Divider for ACLK 00 /1 01 /2 10 /4 11 /8
RSELx	Bits 3-0	Range select. Sixteen different frequency ranges are available. The lowest frequency range is selected by setting RSELx = 0. RSEL3 is ignored when DCOR = 1.

(1) XTS = 1 is not supported in MSP430x20xx and MSP430G2xx devices (see Figure 5-1 and Figure 5-2 for details on supported settings for all devices).

(2) This bit is reserved in the MSP430AFE2xx devices.

BCSCTL2: Basic Clock Control Register 2

7	6	5	4	3	2	1	0
SELMx		DIVMx		SELS	DIVSx		DCOR[1][2]
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

SELMx	Bits 7-6	Select MCLK. These bits select the MCLK source.	
		00	DCOCLK
		01	DCOCLK
		10	XT2CLK when XT2 oscillator present on-chip. LFXT1CLK or VLOCLK when XT2 oscillator not present on-chip.
		11	LFXT1CLK or VLOCLK
DIVMx	Bits 5-4	Divider for MCLK	
		00	/1
		01	/2
		10	/4
		11	/8
SELS	Bit 3	Select SMCLK. This bit selects the SMCLK source.	
		0	DCOCLK
		1	XT2CLK when XT2 oscillator present. LFXT1CLK or VLOCLK when XT2 oscillator not present
DIVSx	Bits 2-1	Divider for SMCLK	
		00	/1
		01	/2
		10	/4
		11	/8
DCOR	Bit 0	DCO resistor select. Not available in all devices. See the device-specific data sheet.	
		0	Internal resistor
		1	External resistor

(1) Does not apply to MSP430x20xx or MSP430x21xx devices.
(2) This bit is reserved in the MSP430AFE2xx devices.

BCSCTL3: Basic Clock Control Register 3

7	6	5	4	3	2	1	0
XT2Sx		LFXT1Sx[1]		XCAPx[2]		XT2OF[3]	LFXT1OF[2]
rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r0	r-(1)

XT2Sx Bits 7-6 XT2 range select. These bits select the frequency range for XT2.

00 0.4- to 1-MHz crystal or resonator
01 1- to 3-MHz crystal or resonator
10 3- to 16-MHz crystal or resonator
11 Digital external 0.4- to 16-MHz clock source

LFXT1Sx Bits 5-4 Low-frequency clock select and LFXT1 range select. These bits select between LFXT1 and VLO when XTS = 0, and select the frequency range for LFXT1 when XTS = 1.

When XTS = 0:
00 32768-Hz crystal on LFXT1
01 Reserved
10 VLOCLK (Reserved in MSP430F21x1 devices)
11 Digital external clock source

When XTS = 1 (Not applicable for MSP430x20xx devices, MSP430G2xx1/2/3)
00 0.4- to 1-MHz crystal or resonator
01 1- to 3-MHz crystal or resonator
10 3- to 16-MHz crystal or resonator
11 Digital external 0.4- to 16-MHz clock source

LFXT1Sx definition for MSP430AFE2xx devices:
00 Reserved
01 Reserved
10 VLOCLK
11 Reserved

XCAPx Bits 3-2 Oscillator capacitor selection. These bits select the effective capacitance seen by the LFXT1 crystal when XTS = 0. If XTS = 1 or if LFXT1Sx = 11 XCAPx should be 00.

00 ~1 pF
01 ~6 pF
10 ~10 pF
11 ~12.5 pF

XT2OF Bit 1 XT2 oscillator fault

0 No fault condition present
1 Fault condition present

LFXT1OF Bit 0 LFXT1 oscillator fault

0 No fault condition present
1 Fault condition present

(1) MSP430G22x0: The LFXT1Sx bits should be programmed to 10b during the initialization and start-up code to select VLOCLK (for more details refer to Digital I/O chapter). The other bits are reserved and should not be altered.
(2) This bit is reserved in the MSP430AFE2xx devices.
(3) Does not apply to MSP430x2xx, MSP430x21xx, or MSP430x22xx devices.

Setting the frequency of the DCO

The frequency of DCOCLK is set by the following functions:

- The four RSELx bits select one of sixteen nominal frequency ranges for the DCO. These ranges are defined in the datasheet.
- The three DCOx bits divide the DCO range selected by the RSELx bits into 8 frequency steps, separated by approximately 10%.
- The five MODx bits, switch between the frequency selected by the DCOx bits and the next higher frequency set by DCOx+1. When DCOx = 07h, the MODx bits have no effect because the DCO is already at the highest setting for the selected RSELx range.

Setting the frequency of the DCO

DCO Frequency

over recommended ranges of supply voltage and operating free-air temperature (unless otherwise noted)

PARAMETER		TEST CONDITIONS	V_{CC}	MIN	TYP	MAX	UNIT
V_{CC}	Supply voltage	RSELx < 14		1.8		3.6	V
		RSELx = 14		2.2		3.6	
		RSELx = 15		3		3.6	
$f_{DCO(0,0)}$	DCO frequency (0, 0)	RSELx = 0, DCOx = 0, MODx = 0	3 V	0.06		0.14	MHz
$f_{DCO(0,3)}$	DCO frequency (0, 3)	RSELx = 0, DCOx = 3, MODx = 0	3 V	0.07		0.17	MHz
$f_{DCO(1,3)}$	DCO frequency (1, 3)	RSELx = 1, DCOx = 3, MODx = 0	3 V		0.15		MHz
$f_{DCO(2,3)}$	DCO frequency (2, 3)	RSELx = 2, DCOx = 3, MODx = 0	3 V		0.21		MHz
$f_{DCO(3,3)}$	DCO frequency (3, 3)	RSELx = 3, DCOx = 3, MODx = 0	3 V		0.30		MHz
$f_{DCO(4,3)}$	DCO frequency (4, 3)	RSELx = 4, DCOx = 3, MODx = 0	3 V		0.41		MHz
$f_{DCO(5,3)}$	DCO frequency (5, 3)	RSELx = 5, DCOx = 3, MODx = 0	3 V		0.58		MHz
$f_{DCO(6,3)}$	DCO frequency (6, 3)	RSELx = 6, DCOx = 3, MODx = 0	3 V	0.54		1.06	MHz
$f_{DCO(7,3)}$	DCO frequency (7, 3)	RSELx = 7, DCOx = 3, MODx = 0	3 V	0.80		1.50	MHz
$f_{DCO(8,3)}$	DCO frequency (8, 3)	RSELx = 8, DCOx = 3, MODx = 0	3 V		1.6		MHz
$f_{DCO(9,3)}$	DCO frequency (9, 3)	RSELx = 9, DCOx = 3, MODx = 0	3 V		2.3		MHz
$f_{DCO(10,3)}$	DCO frequency (10, 3)	RSELx = 10, DCOx = 3, MODx = 0	3 V		3.4		MHz
$f_{DCO(11,3)}$	DCO frequency (11, 3)	RSELx = 11, DCOx = 3, MODx = 0	3 V		4.25		MHz
$f_{DCO(12,3)}$	DCO frequency (12, 3)	RSELx = 12, DCOx = 3, MODx = 0	3 V	4.30		7.30	MHz
$f_{DCO(13,3)}$	DCO frequency (13, 3)	RSELx = 13, DCOx = 3, MODx = 0	3 V	6.00	7.8	9.60	MHz
$f_{DCO(14,3)}$	DCO frequency (14, 3)	RSELx = 14, DCOx = 3, MODx = 0	3 V	8.60		13.9	MHz
$f_{DCO(15,3)}$	DCO frequency (15, 3)	RSELx = 15, DCOx = 3, MODx = 0	3 V	12.0		18.5	MHz
$f_{DCO(15,7)}$	DCO frequency (15, 7)	RSELx = 15, DCOx = 7, MODx = 0	3 V	16.0		26.0	MHz
S_{RSEL}	Frequency step between range RSEL and RSEL+1	$S_{RSEL} = f_{DCO(RSEL+1,DCO)}/f_{DCO(RSEL,DCO)}$	3 V		1.35		ratio
S_{DCO}	Frequency step between tap DCO and DCO+1	$S_{DCO} = f_{DCO(RSEL,DCO+1)}/f_{DCO(RSEL,DCO)}$	3 V		1.08		ratio
	Duty cycle	Measured at SMCLK output	3 V		50		%

Clock Code Example

```
#include <msp430.h>

#define LED BIT7

#define SW1 BIT3                                // 1.5 kHz
#define SW2 BIT4                                //   3 kHz
#define SW3 BIT5                                //  12 kHz

volatile unsigned int i;                        // Volatile to prevent removal

/**
 * @brief
 * Function to take input from 3 switches and change CPU clock accordingly
 **/
void switch_input()
{

    if(!(P1IN & SW1))                   // If SW1 is Pressed
    {
        __delay_cycles(2000);           // Wait 20ms to debounce
        while(!(P1IN & SW1));           // Wait till SW Released
        __delay_cycles(2000);           // Wait 20ms to debounce

        BCSCTL2 &=~ (BIT5 + BIT4);          //Reset VLO divider
        BCSCTL2 |= (BIT5 + BIT4);           //VLO = 12kHz/8 = 1.5kHz
    }


    if(!(P1IN & SW2))                   // If SW is Pressed
    {
        __delay_cycles(2000);           // Wait 20ms to debounce
        while(!(P1IN & SW2));           // Wait till SW Released
        __delay_cycles(2000);           // Wait 20ms to debounce

        BCSCTL2 &=~ (BIT5 + BIT4);          //Reset VLO divider
        BCSCTL2 |= (BIT4);                  //VLO = 12kHz/4 = 3kHz
    }

```

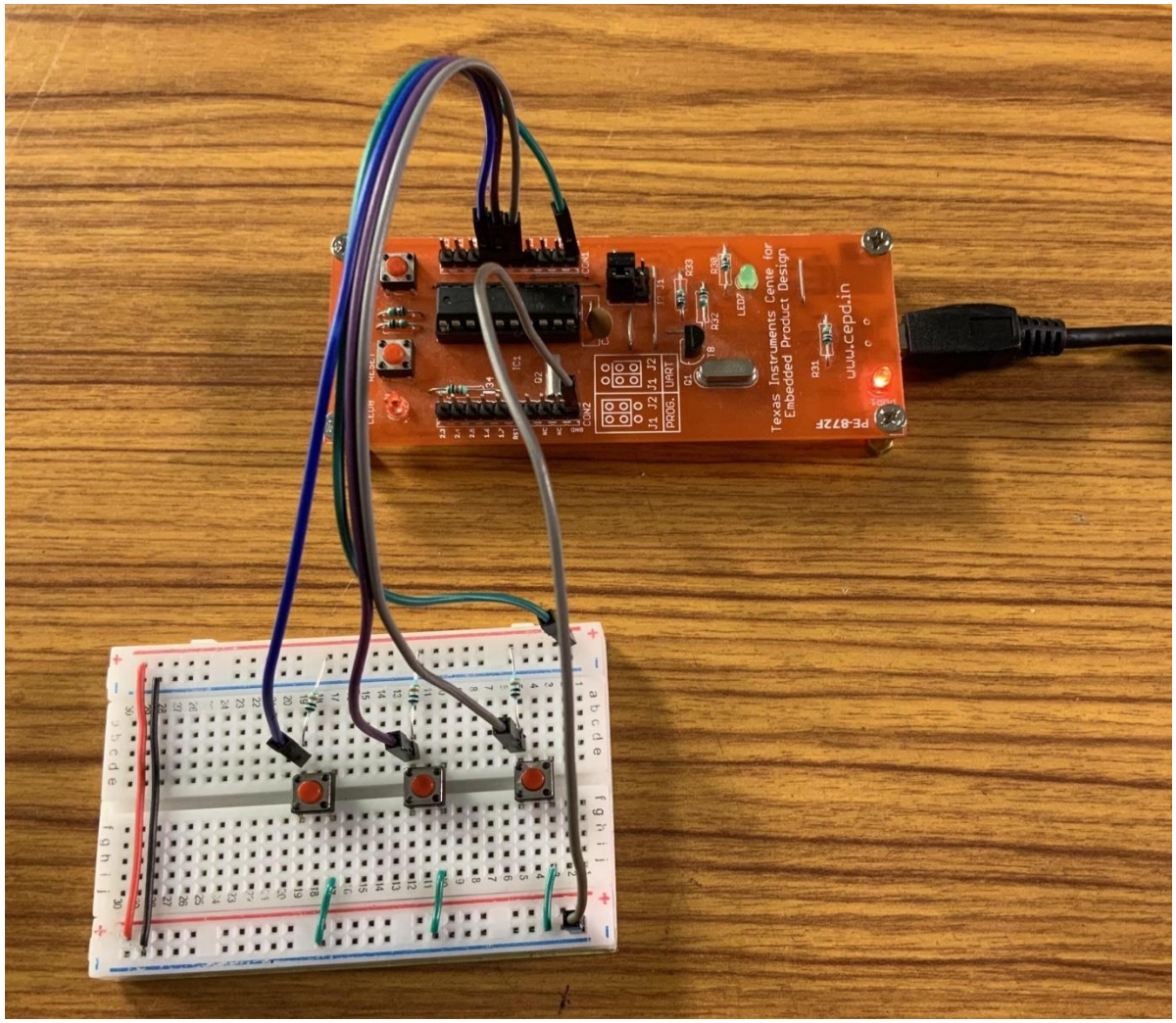
Clock Code Example

```
39      if(!(P1IN & SW3))               // If SW is Pressed
40      {
41
42          __delay_cycles(2000);       // Wait 20ms to debounce
43          while(!(P1IN & SW3));       // Wait till SW Released
44          __delay_cycles(2000);       // Wait 20ms to debounce
45
46          BCSCTL2 &=~ (BIT5 + BIT4);      //VLO = 12kHz/1 = 12kHz
47      }
48
49  }
50
51  /**
52   * @brief
53   * These settings are wrt enabling GPIO on Lunchbox
54   **/
55  void register_settings_for_GPIO()
56  {
57      P1DIR |= LED;                       //P1.7 is set as Output
58      P1DIR &=~ (SW1 + SW2 + SW3);        //P1.3, P1.4, P1.5 are set as Input
59  }
60
61
62  /**
63   * @brief
64   * These settings are w.r.t enabling VLO  on Lunch Box
65   **/
66  void register_settings_for_VLO()
67  {
68      BCSCTL3 |= LFXT1S_2;                        // LFXT1 = VLO
69         do{
70             IFG1 &= ~OFIFG;                      // Clear oscillator fault flag
71             for (i = 50000; i; i--);             // Delay
72           } while (IFG1 & OFIFG);                // Test osc fault flag
73
74         BCSCTL2 |= SELM_3;                       // MCLK = VLO
75  }
76
```

Clock Code Example

```
76
77 /**
78  * main.c
79  */
80 int main(void)
81 {
82
83     WDTCTL = WDTPW | WDTHOLD;                   //Stop watchdog timer
84
85     register_settings_for_VLO();                //Register settings for VLO
86     register_settings_for_GPIO();               //Register settings for GPIO
87
88     while(1)
89     {
90         switch_input();                         //Switch Input
91
92         P1OUT ^= LED;                           //LED Toggle
93         for (i = 100; i > 0; i--);
94
95     }
96 }
```

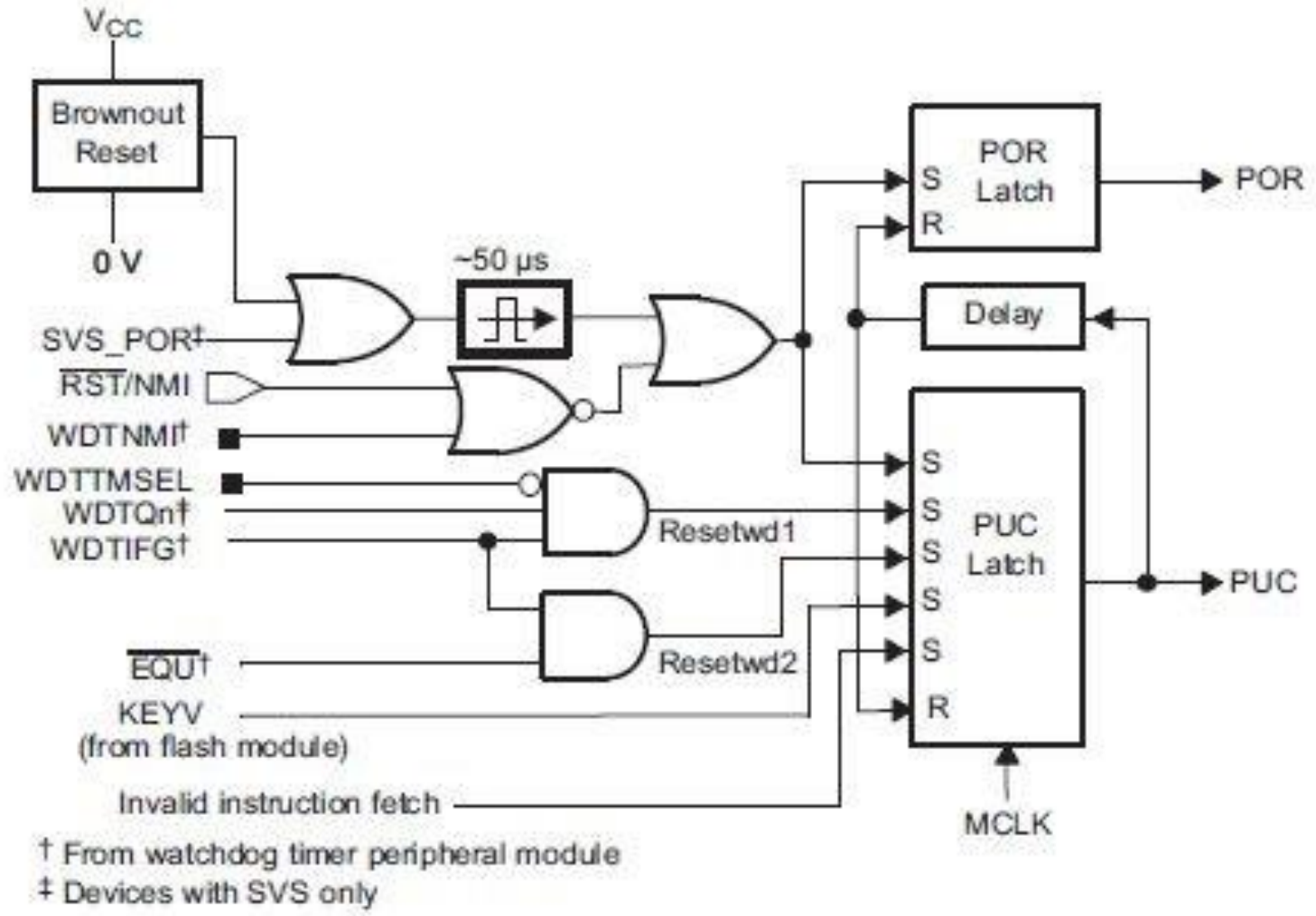
Circuit For The Clock Source Example Code



MSP430 Resets

There are two types of resets:

1. Power on reset (POR)
2. Power up clear (PUC)



Power On Reset

A POR is a device reset and is generated by conditions related to hardware. It occurs in any of the following conditions:

1. Powering up the device
2. Brownout Reset: A POR is raised even when the supply voltage drops to so low a value that the device may not work correctly.
3. A high to low signal on the RST/NMI pin when configured in the reset mode. (User Reset)

Power Up Clear

A PUC is generated by software conditions. It is also always generated when a POR is generated, but a POR is not generated by a PUC. The following events trigger a PUC:

1. A POR signal
2. Watchdog timer expiration when in watchdog mode only
3. Watchdog timer security key violation (An attempt is made to write to the watchdog control register WDTCTL without the correct password 0x05A)
4. A Flash memory security key violation
5. A CPU instruction fetch from the peripheral address range of 0000h to 01FFh

Device Initial Conditions after System Reset

After a POR, the initial MSP430 conditions are:

- The RST/NMI pin is configured in the reset mode.
- I/O pins are switched to input mode.
- Other peripheral modules and registers are initialized as described in the user guide.
- The value is shown under each bit in the description of the registers. For example, rw–0 means that a bit can be both read and written and is initialized to 0 after a PUC. Whereas, rw–(0) shows that a bit is initialized to 0 only after a POR; it retains its value through a PUC.
- Status register (SR) is reset. This means that the device operates at full power, even though it might have been in a low-power mode before the reset occurred.
- The watchdog timer powers up active in watchdog mode.
- Program counter (PC) is loaded with address contained at reset vector location (FFFEh).

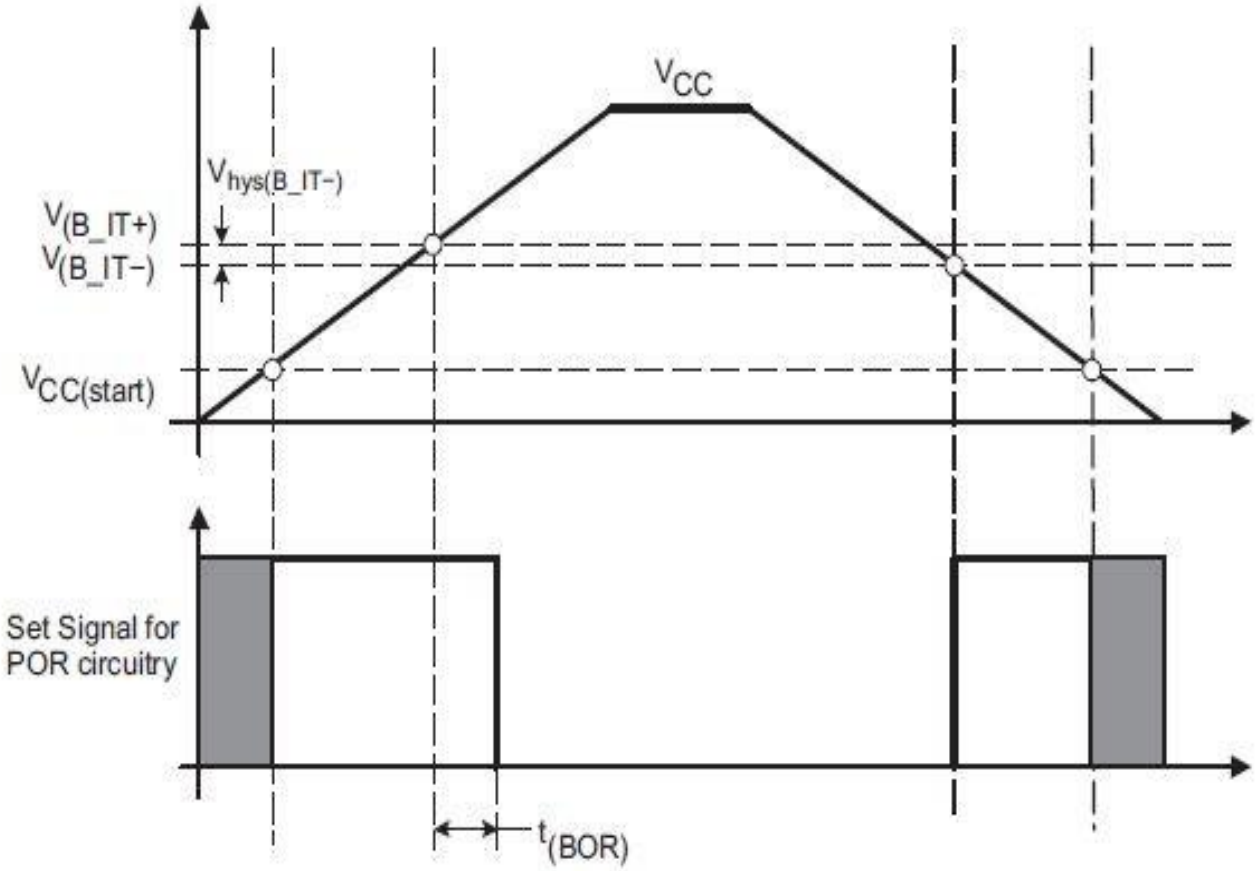
Software Initialisations after System Reset

After a system reset, the software must:

- Initialize the watchdog timer, usually to turn it off
- Configure peripheral modules

Brownout Reset

- The brownout reset circuit detects low supply voltages such as when a supply voltage is applied to or removed from the VCC terminal.
- The brownout reset circuit resets the device by triggering a POR signal when power is applied or removed.



Brownout Reset

- The POR signal becomes active when VCC crosses the VCC(start) level.

  It remains active until VCC crosses the V(B_IT+) threshold and the delay t(BOR) elapses.

- The delay t(BOR) is adaptive being longer for a slow ramping VCC. The hysteresis Vhys(B_ IT-) ensures that the supply voltage must drop below V(B_IT-) to generate another POR signal from the brownout reset circuitry.

Watchdog Timer

- The primary function of the watchdog timer (WDT+) module is to perform a controlled system restart after a software problem occurs such as the program becoming trapped in an unintended, infinite loop
- If the selected time interval expires, a system reset is generated.
- If the watchdog function is not needed in an application, the module can be disabled or configured as an interval timer and can generate interrupts at selected time intervals.

Watchdog Timer Operating Modes

The watchdog timer+ module has two operating modes:

1. Watchdog mode:

- After a PUC, the WDT+ module is configured in the watchdog mode with an initial 32768 cycle reset interval using the DCOCLK.
- The user must setup, halt, or clear the WDT+ prior to the expiration of the initial reset interval or another PUC will be generated.
- When the WDT+ is configured to operate in watchdog mode, either writing to WDTCTL with an incorrect password, or expiration of the selected time interval triggers a PUC.

Watchdog Timer Operating Modes

2. Interval mode

- This mode can be used to provide periodic interrupts.
- In interval timer mode, the WDTIFG flag is set at the expiration of the selected time interval.
- A PUC is not generated in interval timer mode at expiration of the selected timer interval and the WDTIFG enable bit WDTIE remains unchanged.
- When the WDTIE bit and the GIE bit are set, the WDTIFG flag requests an interrupt. The WDTIFG interrupt flag is automatically reset when its interrupt request is serviced, or may be reset by software. The interrupt vector address in interval timer mode is different from that in watchdog mode.

Watchdog Timer Registers

- WDTCTL: Watchdog Timer+ Control Register:
- IE1: SFR Interrupt Enable Register 1
- IFG1: SFR Interrupt Flag Register 1

WDTCTL Register

- WDTCTL is a 16-bit, password-protected, read/write register.
- Any read or write access must use word instructions and write accesses must include the write password 05Ah in the upper byte.
- Any write to WDTCTL with any value other than 05Ah in the upper byte is a security key violation and triggers a PUC system reset regardless of timer mode.
- Any read of WDTCTL reads 069h in the upper byte.

WDTCTL Register

15	14	13	12	11	10	9	8
WDTPW, Read as 069h Must be written as 05Ah							

7	6	5	4	3	2	1	0
WDTHOLD	**WDTNMIES**	**WDTNMI**	**WDTTMSEL**	**WDTCNTCL**	**WDTSSEL**	**WDTISx**	
rw-0	rw-0	rw-0	rw-0	r0(w)	rw-0	rw-0	rw-0

WDTPW Bits 15-8 Watchdog timer+ password. Always read as 069h. Must be written as 05Ah, or a PUC is generated.

WDTHOLD Bit 7 Watchdog timer+ hold. This bit stops the watchdog timer+. Setting WDTHOLD = 1 when the WDT+ is not in use conserves power.

- 0 Watchdog timer+ is not stopped
- 1 Watchdog timer+ is stopped

WDTNMIES Bit 6 Watchdog timer+ NMI edge select. This bit selects the interrupt edge for the NMI interrupt when WDTNMI = 1. Modifying this bit can trigger an NMI. Modify this bit when WDTIE = 0 to avoid triggering an accidental NMI.

- 0 NMI on rising edge
- 1 NMI on falling edge

WDTNMI Bit 5 Watchdog timer+ NMI select. This bit selects the function for the $\overline{\text{RST}}$/NMI pin.

- 0 Reset function
- 1 NMI function

WDTTMSEL Bit 4 Watchdog timer+ mode select

- 0 Watchdog mode
- 1 Interval timer mode

WDTCNTCL Bit 3 Watchdog timer+ counter clear. Setting WDTCNTCL = 1 clears the count value to 0000h. WDTCNTCL is automatically reset.

- 0 No action
- 1 WDTCNT = 0000h

WDTSSEL Bit 2 Watchdog timer+ clock source select

- 0 SMCLK
- 1 ACLK

WDTISx Bits 1-0 Watchdog timer+ interval select. These bits select the watchdog timer+ interval to set the WDTIFG flag and/or generate a PUC.

- 00 Watchdog clock source /32768
- 01 Watchdog clock source /8192
- 10 Watchdog clock source /512
- 11 Watchdog clock source /64

IE1: SFR Interrupt Enable Register 1

- WDTIE bit enables the WDTIFG interrupt for interval timer mode. It is not necessary to set this bit for watchdog mode.

Address	7	6	5	4	3	2	1	0
00h			ACCVIE	NMIIE			OFIE	WDTIE
			rw-0	rw-0			rw-0	rw-0

WDTIE	Watchdog Timer interrupt enable. Inactive if watchdog mode is selected. Active if Watchdog Timer is configured in interval timer mode.
OFIE	Oscillator fault interrupt enable
NMIIE	(Non)maskable interrupt enable
ACCVIE	Flash access violation interrupt enable

IFG1: SFR Interrupt Flag Register 1

Address	7	6	5	4	3	2	1	0
02h				NMIIFG	RSTIFG	PORIFG	OFIFG	WDTIFG
				rw-0	rw-(0)	rw-(1)	rw-1	rw-(0)

WDTIFG	Set on watchdog timer overflow (in watchdog mode) or security key violation. Reset on V_{CC} power-on or a reset condition at the $\overline{RST}$/NMI pin in reset mode.
OFIFG	Flag set on oscillator fault.
PORIFG	Power-On Reset interrupt flag. Set on V_{CC} power-up.
RSTIFG	External reset interrupt flag. Set on a reset condition at $\overline{RST}$/NMI pin in reset mode. Reset on V_{CC} power-up.
NMIIFG	Set via $\overline{RST}$/NMI pin

Stopping of the watchdog timer

The watchdog timer is stopped by the following statement:

WDTCTL = WDTPW + WDTHOLD;

This instruction sets the password (WDTPW) as 5Ah and the bit to stop the timer (WDTHOLD).

Determining the Reset Source: Code Example

```
#include <msp430.h>

#define  LED1 BIT0                                  // main() has started
#define  LED2 BIT1                                  // program is executing
#define  LEDA BIT2                                  // POR
#define  LEDB BIT3                                  // WDT
#define  LEDC BIT4                                  // RST

/**
 * @brief
 * These settings are wrt enabling GPIO on Lunchbox
 **/
void register_settings_for_GPIO()
{
    P1DIR |= (LED1 + LED2 + LEDA + LEDB + LEDC);        // P1.0 to P1.4 as output

    P1OUT |= LED1;                                  // LED1 representing main() is set

    P1OUT &=~ (LED2 + LEDA + LEDB + LEDC);      // Turning all other LEDs OFF
}

/**
 * @brief
 * These settings are w.r.t check source of reset on Lunch Box
 **/
void checking_reset_source()
{
    if (IFG1 & PORIFG)                          // Check for Power on Reset flag (POR)
    {
        P1OUT|= LEDA;                           // Turning LEDA On
        P1OUT &= ~LEDB;                         // Turning LEDB Off
        P1OUT &= ~LEDC;                         // Turning LEDC Off

        /* Settings for Watchdog Timer register
                Giving Watchdog Timer Password
                Clearing Watchdog counter to 0000h
                Watchdog Source select --> ACLK
                Watchdog Timer interval set to generate watchdog reset at 2 secs
        */
        WDTCTL = (WDTPW + WDTCNTCL + WDTSSEL + WDTIS0);

        IFG1 &=~ PORIFG;                        // Clearing Power on Reset flag
    }
```

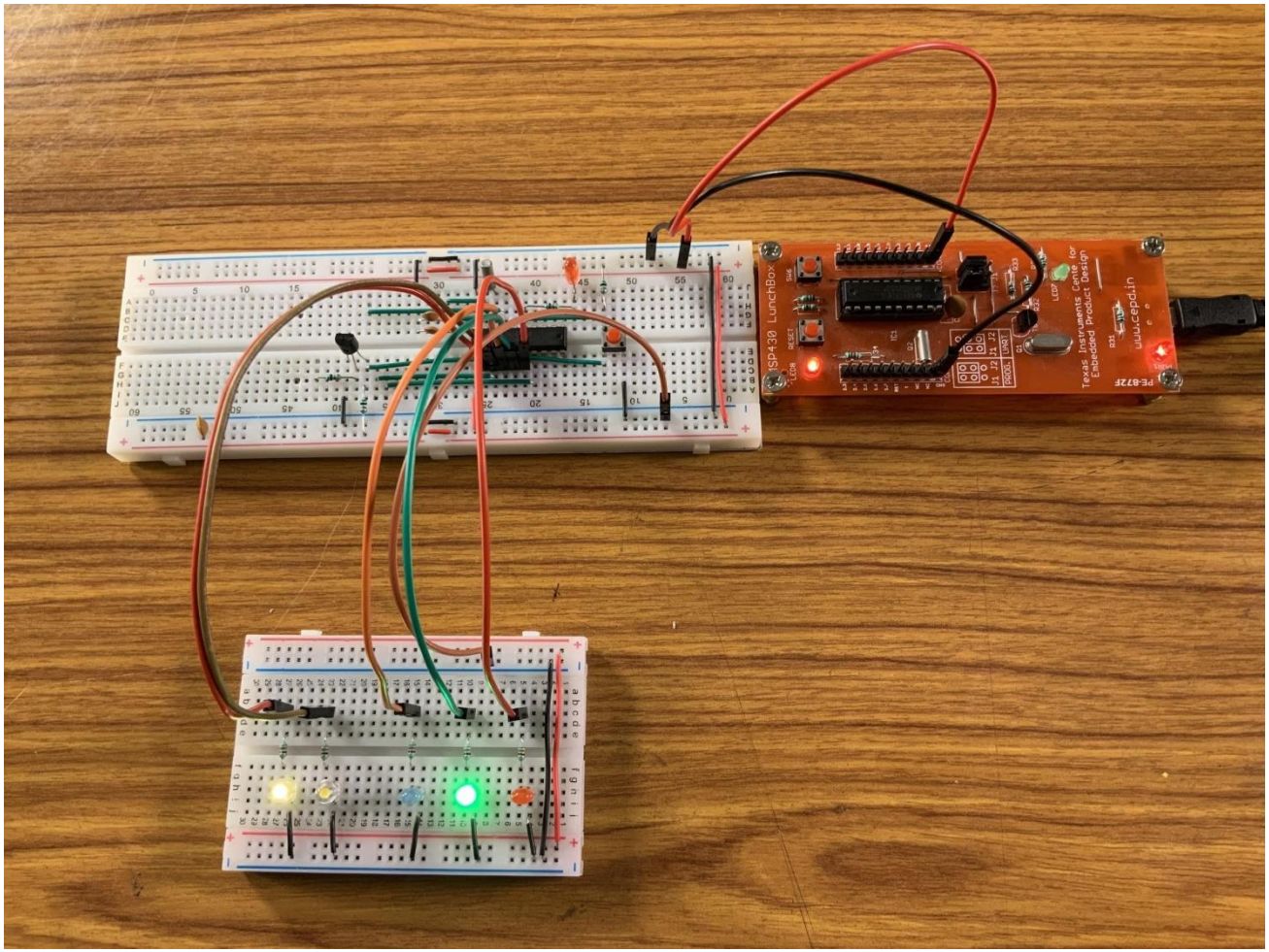
Determining the Reset Source: Code Example

```
43      }
44
45      else if (IFG1 & RSTIFG)                 // Check for External Reset flag (RST)
46      {
47          P1OUT|= LEDC;                       // Turning LEDC On
48          P1OUT &= ~LEDA;                     // Turning LEDA Off
49          P1OUT &= ~LEDB;                     // Turning LEDB Off
50
51          /* Settings Watchdog Timer register
52                  Giving Watchdog Timer Password
53                  Clearing Watchdog counter to 0000h
54                  Watchdog Source select --> ACLK
55                  Watchdog Timer interval set to generate watchdog reset at 2 secs
56          */
57          WDTCTL = (WDTPW + WDTCNTCL + WDTSSEL + WDTIS0);
58
59          IFG1 &=~ RSTIFG;                    // Clearing External Reset flag
60      }
61
62      else if (IFG1 & WDTIFG)                 // Check for Watch Dog Reset flag (WDT)
63      {
64          P1OUT|= LEDB;                       // Turning LEDB On
65          P1OUT &= ~LEDA;                     // Turning LEDA Off
66          P1OUT &= ~LEDC;                     // Turning LEDC Off
67
68          IFG1 &=~ PORIFG;                    // Clearing Power on Reset flag
69          IFG1 &=~ RSTIFG;                    // Clearing External Reset flag
70      }
71
72  }
```

Determining the Reset Source: Code Example

```
73
74 /*@brief entry point for the code*/
75 void main(void)
76 {
77       WDTCTL = WDTPW | WDTHOLD;    // stop watch dog timer
78       volatile unsigned int i;
79
80       BCSCTL1 |= DIVA_3;             // Dividing ACLK by 8, 32768Hz/8 = 4096Hz
81
82       register_settings_for_GPIO();
83
84       /* This loop checks for Oscillator fault flag to reset means
85          it delays execution until external crystal is Power On  */
86
87       do
88       {
89          IFG1 &= ~OFIFG;                          // Clear oscillator fault flag
90          for (i = 10000; i ; i--);                // Delay, minimum value of i = 5000
91       }  while (IFG1 & OFIFG);                    // Test osc fault flag
92
93       checking_reset_source();
94
95       while(1)
96       {
97           P1OUT ^= LED2;                         // Toggle LED
98           __delay_cycles(1000000);
99       }
100 }
101
```

Circuit For The Reset Source Example Code



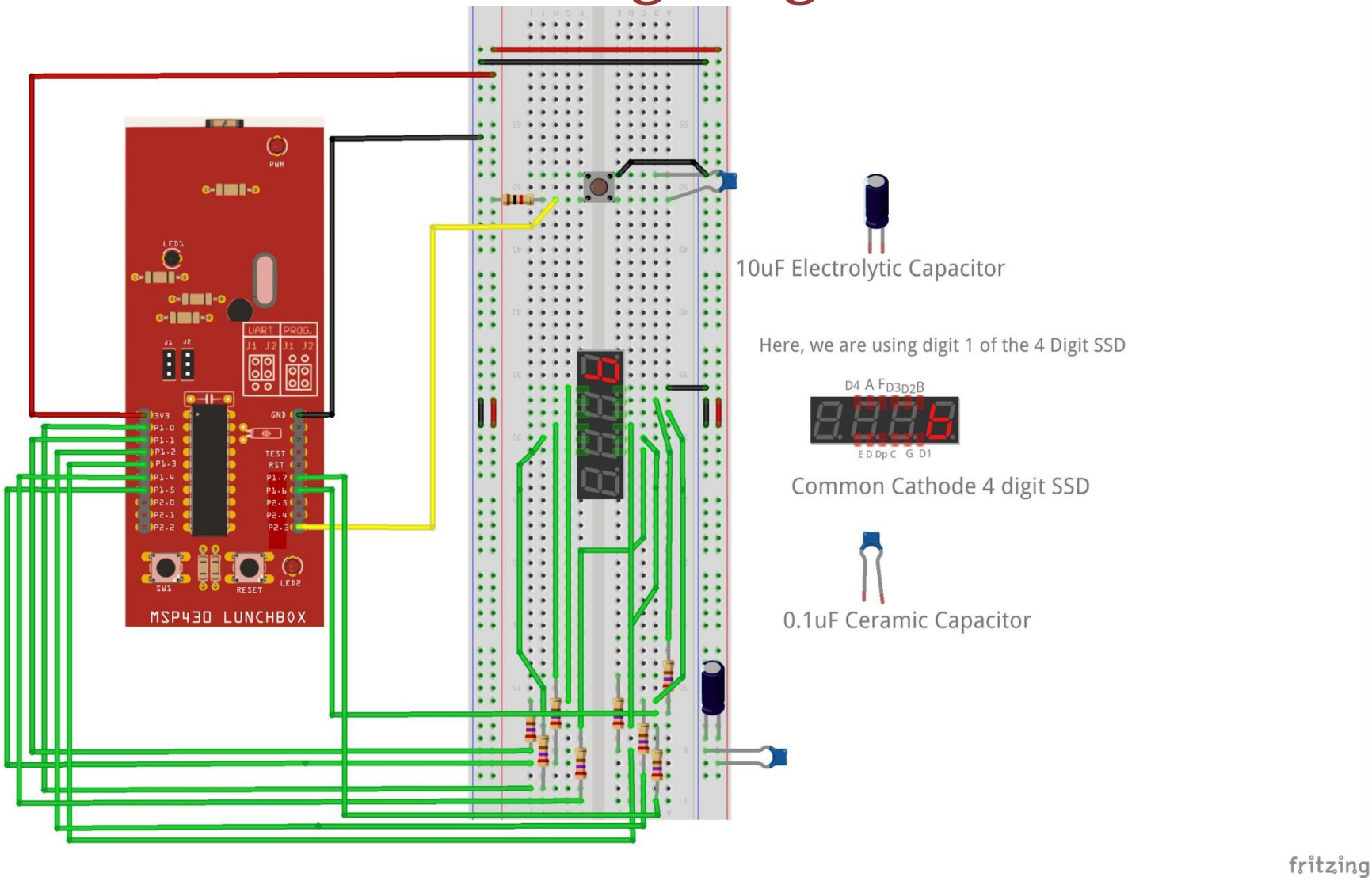
Interfacing SSD with MSP430

HelloSSD:

This example code is used to display a hexadecimal (0 to F) up counter on a single digit SSD(Common Cathode). The count increases on every press of an external switch.

Interfacing SSD with MSP430

Fritzing Diagram



Code Example: HelloSSD

```
1 #include <msp430.h>
2
3 #define SW      BIT3                    // Switch -> P2.3
4
5 // Define Pin Mapping of 7-segment Display
6 // Segments are connected to P1.0 - P1.7
7 #define SEG_A   BIT0
8 #define SEG_B   BIT1
9 #define SEG_C   BIT2
10 #define SEG_D   BIT3
11 #define SEG_E   BIT4
12 #define SEG_F   BIT5
13 #define SEG_G   BIT6
14 #define SEG_DP  BIT7
15
16 // Define each digit according to truth table
17 #define D0  (SEG_A + SEG_B + SEG_C + SEG_D + SEG_E + SEG_F)
18 #define D1  (SEG_B + SEG_C)
19 #define D2  (SEG_A + SEG_B + SEG_D + SEG_E + SEG_G)
20 #define D3  (SEG_A + SEG_B + SEG_C + SEG_D + SEG_G)
21 #define D4  (SEG_B + SEG_C + SEG_F + SEG_G)
22 #define D5  (SEG_A + SEG_C + SEG_D + SEG_F + SEG_G)
23 #define D6  (SEG_A + SEG_C + SEG_D + SEG_E + SEG_F + SEG_G)
24 #define D7  (SEG_A + SEG_B + SEG_C)
25 #define D8  (SEG_A + SEG_B + SEG_C + SEG_D + SEG_E + SEG_F + SEG_G)
26 #define D9  (SEG_A + SEG_B + SEG_C + SEG_D + SEG_F + SEG_G)
27 #define DA  (SEG_A + SEG_B + SEG_C + SEG_E + SEG_F + SEG_G)
28 #define DB  (SEG_C + SEG_D + SEG_E + SEG_F + SEG_G)
29 #define DC  (SEG_A + SEG_D + SEG_E + SEG_F)
30 #define DD  (SEG_B + SEG_C + SEG_D + SEG_E + SEG_G)
31 #define DE  (SEG_A + SEG_D + SEG_E + SEG_F + SEG_G)
32 #define DF  (SEG_A + SEG_E + SEG_F + SEG_G)
33
34
```

```
34
35 // Define mask value for all digit segments except DP
36 #define DMASK   ~(SEG_A + SEG_B + SEG_C + SEG_D + SEG_E + SEG_F + SEG_G)
37
38 // Store digits in array for display
39 const unsigned int digits[16] = {D0, D1, D2, D3, D4, D5, D6, D7, D8, D9, DA, DB, DC, DD, DE, DF};
40
41 volatile unsigned int i = 0;
42
43 /*@brief entry point for the code*/
44 void main(void) {
45     WDTCTL = WDTPW | WDTHOLD;           //! Stop Watch dog
46
47     // Initialize 7-segment pins as Output
48     P1DIR |= (SEG_A + SEG_B + SEG_C + SEG_D + SEG_E+ SEG_F + SEG_G + SEG_DP);
49
50     P2DIR &= ~SW;                       // Set SW pin -> Input
51
52     while(1)
53     {
54         if(!(P2IN & SW))            // If SW is Pressed
55         {
56             __delay_cycles(20000);      //Delay to avoid Switch Bounce
57             while(!(P2IN & SW));        // Wait till SW Released
58             __delay_cycles(20000);      //Delay to avoid Switch Bounce
59             i++;                        //Increment count
60             if(i>15)
61             {
62                 i=0;
63             }
64         }
65         P1OUT = (P1OUT & DMASK) + digits[i];    // Display current digit
66     }
67 }
```

Low Power Modes in MSP430

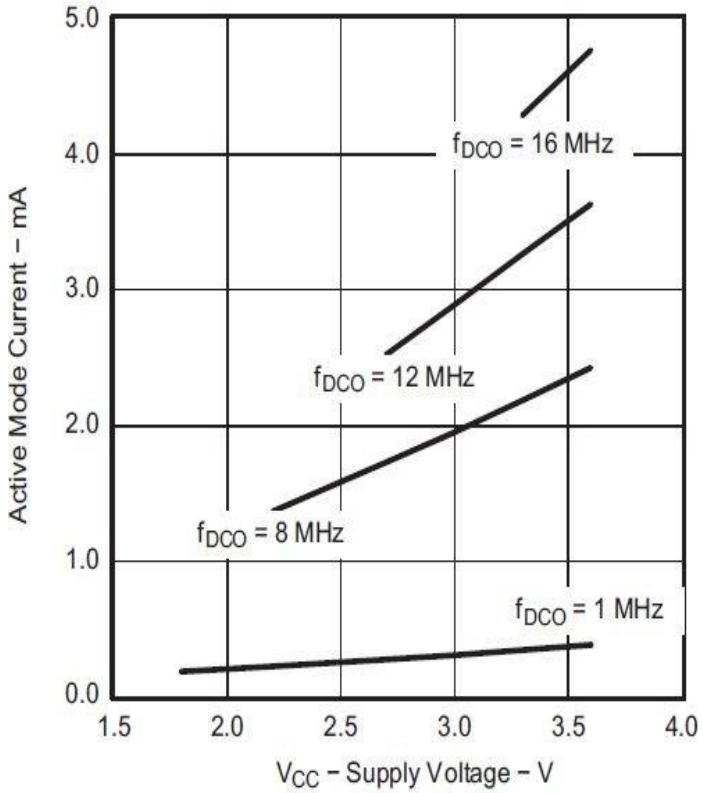
The MSP430 was designed primarily for low power applications and this is reflected in a range of low-power modes of operation.

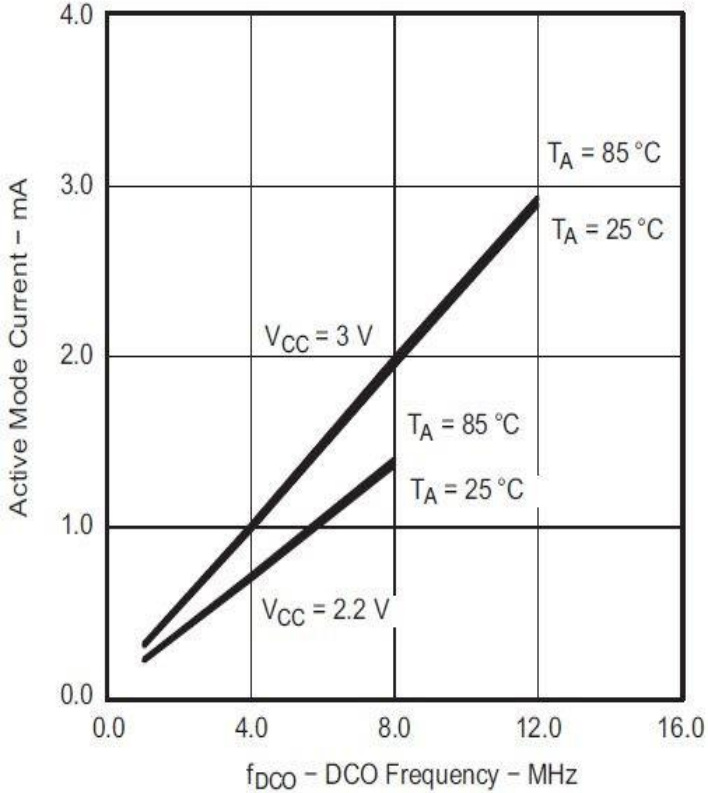
OPERATING MODES:

- Active Mode
- LPM0
- LPM1
- LPM2
- LPM3
- LPM4

Active Mode

- All clocks are active.
- SUPPLY CURRENT(at 1MHz):
  - 230uA(at 2.2V)
  - 330uA(at 3.0V)





LPM0

- CPU is disabled.
- ACLK and SMCLK remain active, MCLK is **disabled**.
- Supply current(at 1 MHz) : 56uA(at 2.2V)

LPM1

- CPU is disabled.
- ACLK and SMCLK remain active, MCLK is disabled.
- DCO and DC generator is disabled if DCO is not used for SMCLK

LPM2

- CPU is disabled.
- MCLK and SMCLK are disabled.
- DCO's DC generator remains enabled.
- ACLK remains active.
- Supply Current : 22uA(at 2.2V)

LPM3

- CPU is disabled.
- MCLK and SMCLK are disabled.
- DCO's DC generator is disabled.
- ACLK remains active.
- Supply Current : 0.5uA(at 2.2V)

LPM4

- CPU is disabled.
- ACLK is disabled.
- MCLK and SMCLK are disabled.
- DCO's DC generator is disabled.
- Crystal oscillator is stopped.
- Supply Current : 0.1uA(at 2.2V)

Controlling Low Power Modes

- LPM operations are controlled through 4 bits of the Status Register: SCG0, SCG1, CPUOFF and OSCOFF.
- All these bits are cleared in active mode and particular combinations are set for each LPM.

Controlling Low Power Modes

SCG1	SCG0	OSCOFF	CPUOFF	Mode	CPU and Clocks Status
0	0	0	0	Active	CPU is active, all enabled clocks are active
0	0	0	1	LPM0	CPU, MCLK are disabled, SMCLK, ACLK are active
0	1	0	1	LPM1	CPU, MCLK are disabled. DCO and DC generator are disabled if the DCO is not used for SMCLK. ACLK is active.
1	0	0	1	LPM2	CPU, MCLK, SMCLK, DCO are disabled. DC generator remains enabled. ACLK is active.
1	1	0	1	LPM3	CPU, MCLK, SMCLK, DCO are disabled. DC generator disabled. ACLK is active.
1	1	1	1	LPM4	CPU and all clocks disabled

- Two ways to put MSP430 into LPM Mode 4, for example, are:
  - __low_power_mode_4(); //This also enables the GIE bit.
  - __bis_SR_register(LPM4_bits + GIE); //+GIE is used to enable interrupts.

Interrupts and LPM

- MSP430 is designed to stay in a Low Power Mode for most of the time.
- The main function **generally** configures the peripherals, enables interrupts, puts the MSP430 into LPM, and plays no further role.
- MSP430 is woken up only when **an interrupt comes**.
- An LPM is suspended whenever an enabled interrupt is serviced.
- On entering the ISR, the SR is stored on stack and the CPUOFF, SCG1, OSCOFF bits are automatically reset.
- On exiting the ISR, the original SR is popped from the stack and the previous operating mode is restored, so that the LPM is resumed after the ISR.

Ultra Low Power Mode Advisor

- ULP Mode Advisor is integrated into CCS.
- It checks your code against a thorough checklist to achieve the lowest power possible and provides detailed notifications and remarks

Code Example: HelloLPM

```
1 #include <msp430.h>
2
3 #define SW      BIT3                    // Switch -> P1.3
4 #define RED     BIT7                      // Red LED -> P1.7
5
6 /*@brief entry point for the code*/
7 void main(void) {
8     WDTCTL = WDTPW | WDTHOLD;           // Stop Watch dog timer
9
10    P1DIR |= RED;                       // Set LED pin -> Output
11    P1OUT &= ~RED;                      // Turn RED LED off
12
13    P1DIR &= ~SW;                       // Set SW pin -> Input
14    P1IES &= ~SW;                       // Select Interrupt on Rising Edge
15    P1IE |= SW;                         // Enable Interrupt on SW pin
16
17    unsigned int i;
18
19    while(1)
20    {
21        __bis_SR_register(LPM4_bits + GIE); // Enter LPM4 and Enable CPU Interrupt
22
23        P1OUT ^= RED;                       // Toggle RED LED
24        for(i=0; i<20000; i++);
25    }
26 }
27
28 /*@brief entry point for switch interrupt*/
29 #pragma vector=PORT1_VECTOR
30 __interrupt void Port_1(void)
31 {
32    __bic_SR_register_on_exit(LPM4_bits + GIE);     // Exit LPM4 on return to main
33    P1IFG &= ~SW;                                   // Clear SW interrupt flag
34 }
```

MSP430 Nomenclature

MSP430F2618ATZQWT-EP

Processor Family

CC = Embedded RF Radio
MSP = Mixed Signal Processor
XMS = Experimental Silicon

430 MCU Platform

Device Type

Memory Type
C = ROM
F = Flash
FR = FRAM
G = Flash (Value Line)
L = No non-volatile memory

Specialized Application
FG = Flash Medical
CG = ROM Medical
FE = Flash Energy Meter
FW = Flash Electronic Flow Meters
AFE = Analog Front End
BT = Pre-programmed with Bluetooth
BQ = Contactless Power

Generation

1 series = up to 8 MHz
2 series = up to 16 MHz
3 series = Legacy OTP
4 series = up to 16 MHz w/ LCD
5 series = up to 25 MHz
6 series = up to 25 MHz w/ LCD
0 = Low Voltage Series

Family

Series and Device Number

Optional: A = revision

Optional: Temperature range

S = 0°C to 50°C
I = -40°C to 85°C
T = -40°C to 105°C

Packaging

www.ti.com/packaging

Optional: Distribution format

T = Small Reel (7-in)
R = Large Reel (11-in)
No markings = Tube or Tray

Optional: Additional features

*-Q1 = Automotive qualified
*-EP = Enhanced product (-40°C to 125°C)
*-HT = Extreme temp parts (-55°C to 150°C)

References:

- Textbooks prescribed in the university syllabus.
- As per the presentation slides received during Online Faculty Orientation Workshop under the aegis of BOS (E&TC), SPPU, Pune for syllabus discussions.